Computer Vision

ACTL3143 & ACTL5111 Deep Learning for Actuaries
Patrick Laub

UNSW SYDNEY

Lecture Outline

- **Introduction**
- The Convolution Operation
- Convolutional Neural Networks
- Chinese Character Recognition Dataset
- Training Our Models
- Hyperparameter Optimisation
- Transfer Learning

What is Computer Vision?

> "Little by little, we're giving sight to the machines. First, we teach them to see. Then, they help us to see better."
>
> — Fei-Fei Li, creator of ImageNet

Getting computers to *extract meaning from images and video* — and act on what they see.

- *Image classification* — one label per image (benign vs malignant lesion, pass/fail parts).
- *Object detection* — locate and label many objects at once (pedestrians and traffic lights for a self-driving car).
- *Image segmentation* — label *every pixel* (outlining a tumour in a scan).
- *Facial recognition* — verify or identify a person (phone unlock, passport gates).

Source: Fei-Fei Li (2015), How we're teaching computers to understand pictures, TED2015.

Computer vision for actuaries

- *Claims automation* — estimate motor repair costs from photos of the damage.
- *Property underwriting* — score roof condition and bushfire exposure from aerial imagery.
- *Health & life* — predict cardiovascular risk factors from retinal scans.
- *Fraud detection* — flag reused, stock, or AI-altered claim images.

For an actuary the practical upshot is less about any individual architecture and more about *transfer learning*: a CNN trained on millions of images can be downloaded and reused on your own (much smaller) dataset.

Facial analytics and life insurance

How your selfie could affect your life insurance

Originally published April 20, 2017 at 7:27 am

FILE – In this Tuesday, Feb. 28, 2017, file photo, Fidelio Desbradel and his wife, Leonor Desbradel, of the Dominican Republic, take a selfie in front of a Tulip Magnolia tree in Washington. A selfie reveals more than whether it's a... (AP Photo/Cliff Owen, File) More

Share story

Share

Email

By BARBARA MARQUAND
The Associated Press

A selfie reveals more than whether it's a good hair day. Facial lines and contours, droops and dark spots could indicate how well you're aging, and, when paired with other data, could someday help determine whether you qualify for life insurance.

Since November 2015, created AI to predict mortality from photos.

COVENTRY
REDEFINING INSURANCE®

Submit a Policy

Home Who We Are What We Do News Careers

Flaws in Lapetus Life Expectancy Estimates Exposed in New Study Following Olshansky-Buerger Debate at LISA Conference

Posted on February 10, 2025 at 4:37 pm

The Study Measures Lapetus's Actual-to-Expected Results

A comprehensive new study conducted by Professors Daniel Bauer and Nan Zhu has revealed serious flaws in the life expectancy estimates produced by Lapetus Solutions, Inc., highlighting that the company significantly understates the actual life expectancies of insureds. The findings raise serious concerns about the accuracy of Lapetus's mortality predictions, which could have serious repercussions for life settlement investors.

Lapetus shut down in August 31, 2025.

Source: Seattle Times & Coventry.

Imports needed for these demos

```
import random
from pathlib import Path

import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from matplotlib.image import imread
from matplotlib.pyplot import imshow
import numpy as np
import pandas as pd
from PIL import Image

import keras
from keras.models import Sequential, Model
from keras.layers import (Dense, Input, Rescaling, Flatten,
    Conv2D, MaxPooling2D, GlobalAveragePooling2D)
from keras.callbacks import EarlyStopping
from keras.utils import plot_model

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from directory_tree import DisplayTree
import optuna
```

For PIL, you'll need to `pip install Pillow`.

Shapes of data

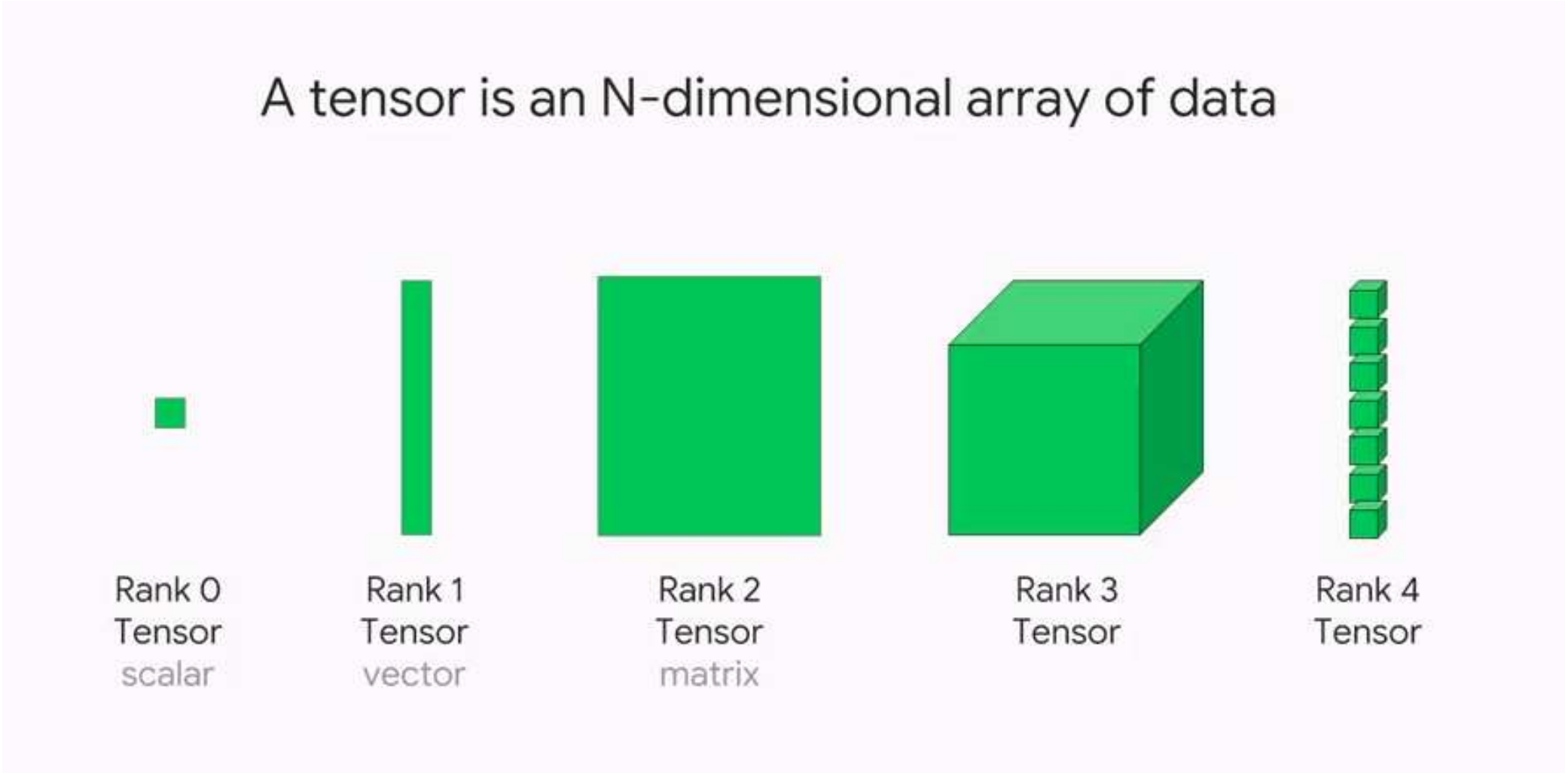
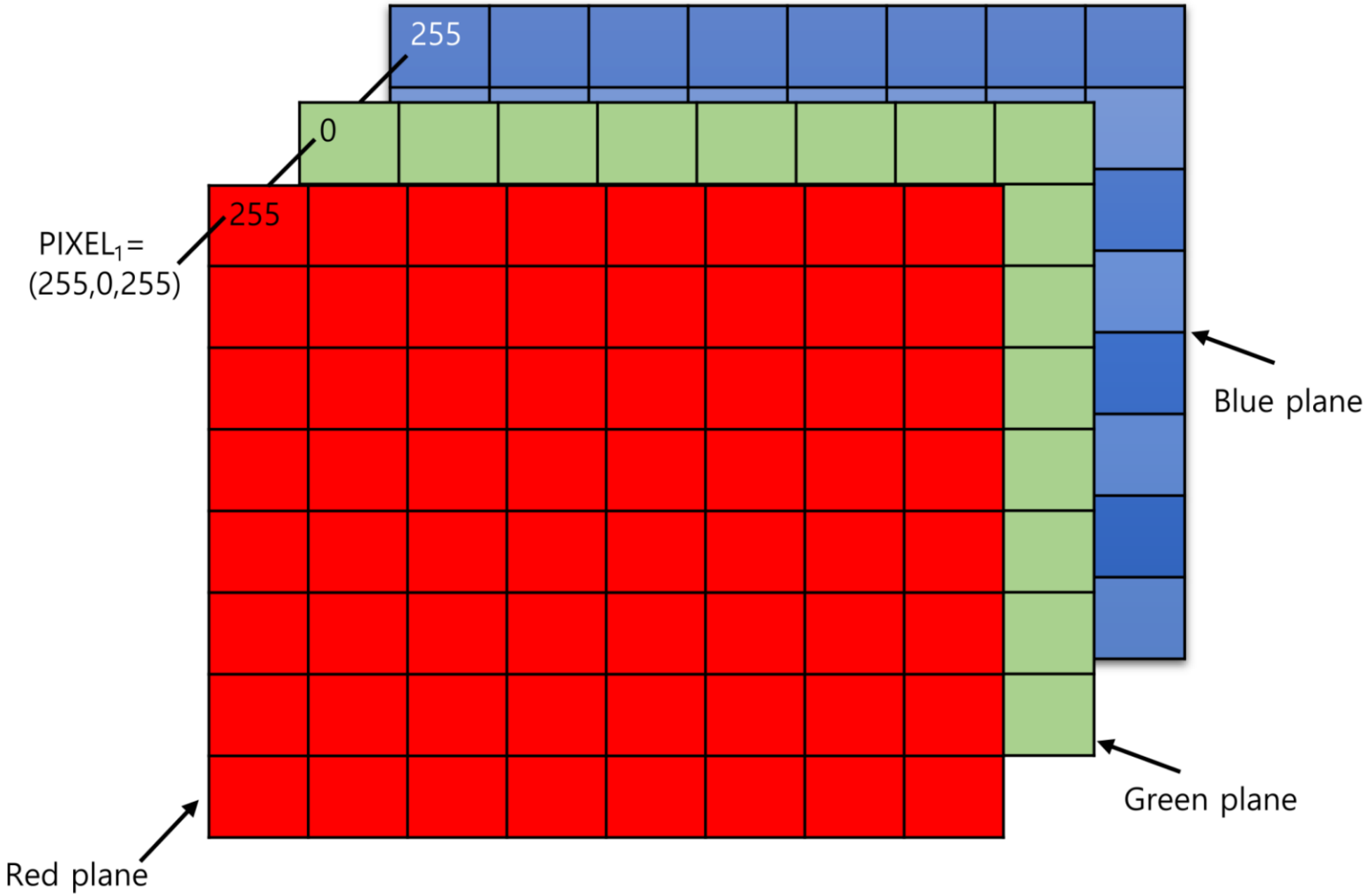


Illustration of tensors of different rank.

Source: Paras Patidar (2019), Tensors — Representation of Data In Neural Networks, Medium article.

Shapes of photos



A photo is a rank 3 tensor.

Source: Kim et al (2021), Data Hiding Method for Color AMBTC Compressed Images Using Color Difference, Applied Sciences.

How the computer sees them

```
1 img1 = imread('images/pu.gif'); img2 = imread('images/pl.gif')
2 img3 = imread('images/pr.gif'); img4 = imread('images/pg.bmp')
3 f"Shapes are: {img1.shape}, {img2.shape}, {img3.shape}, {img4.shape}."
```

```
'Shapes are: (16, 16, 3), (16, 16, 3), (16, 16, 3), (16, 16, 3).'
```

```
1 print(img1)
```

```
[[[  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]]
```

```
1 print(img2)
```

```
[[[  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [255 255   0]
  [255 255   0]
  [255 255   0]
  [255 255   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]]
```

```
1 print(img3)
```

```
[[[  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [255 255   0]
  [255 255   0]
  [255 255   0]
  [255 255   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]]
```

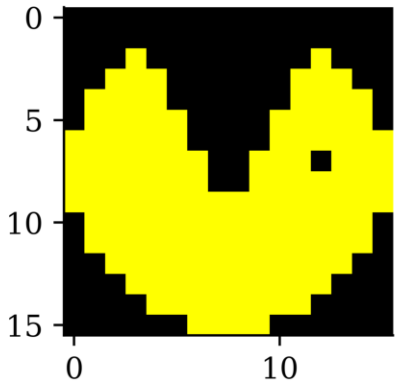
```
1 print(img4)
```

```
[[[  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [255 163 177]
  [255 163 177]
  [255 163 177]
  [255 163 177]
  [255 163 177]
  [255 163 177]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]
  [  0   0   0]]
```

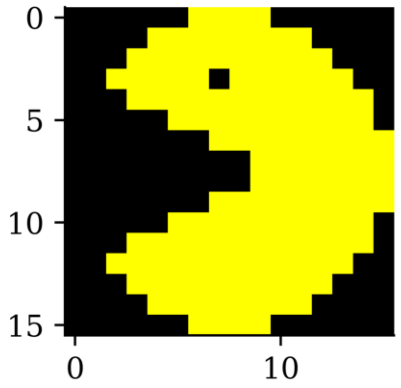
UNSW SYDNEY

How we see them

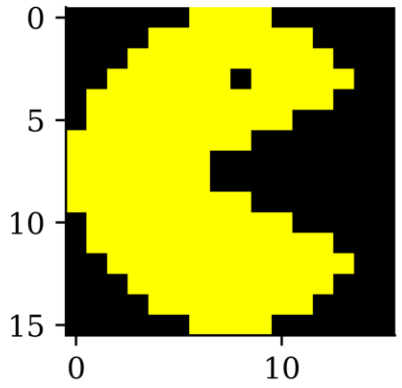
```
1 imshow(img1);
```



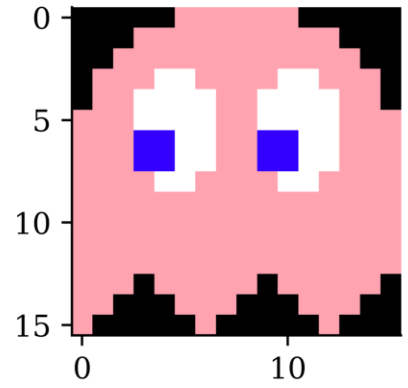

```
1 imshow(img2);
```




```
1 imshow(img3);
```




```
1 imshow(img4);
```



Why is 255 special?

Each pixel's colour intensity is stored in one byte.

One byte is 8 bits, so in binary that is 00000000 to 11111111.

The largest *unsigned* number this can be is $2^8 - 1 = 255$.

```
1 np.array([0, 1, 255, 256]).astype(np.uint8)
```

```
array([  0,   1, 255,   0], dtype=uint8)
```

If you had *signed* numbers, this would go from -128 to 127.

```
1 np.array([-128, 1, 127, 128]).astype(np.int8)
```

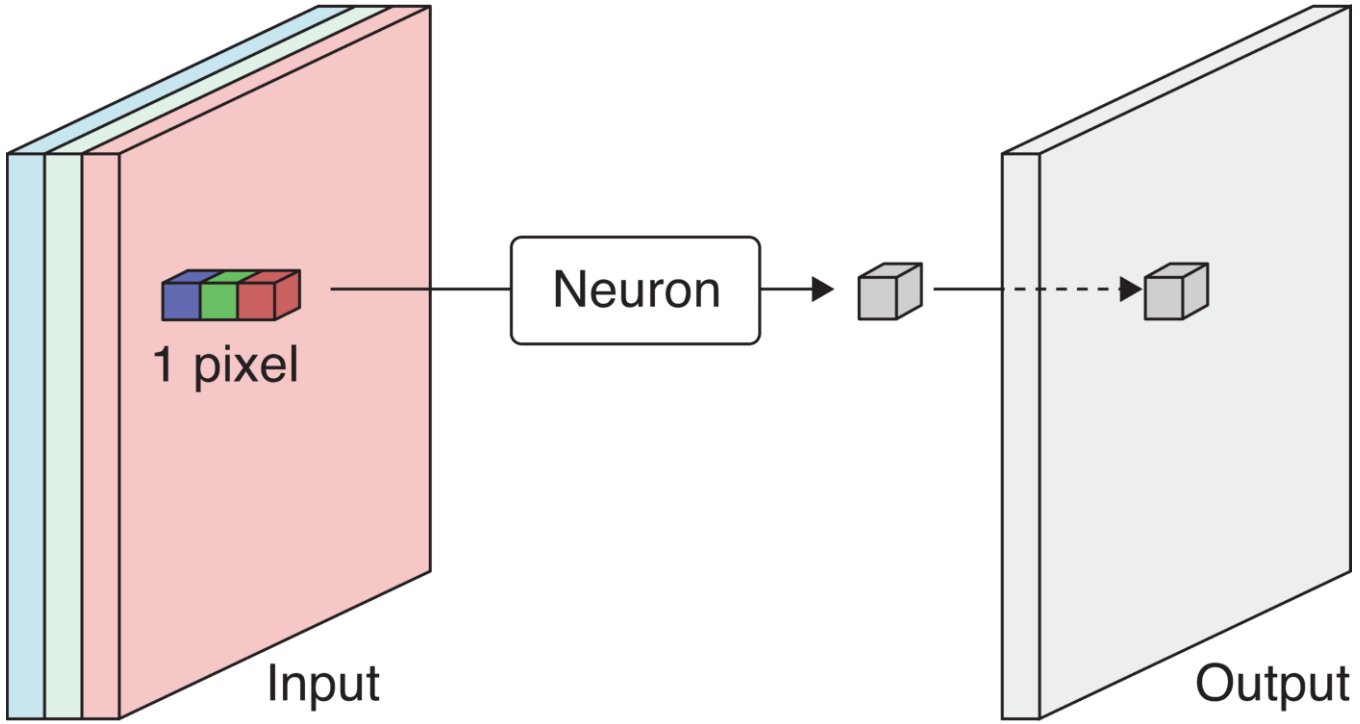
```
array([-128,    1,  127, -128], dtype=int8)
```

Alternatively, *hexadecimal* numbers are used. E.g. 10100001 is split into 1010 0001, and 1010=A, 0001=1, so combined it is 0xA1.

Lecture Outline

- Introduction
- **The Convolution Operation**
- Convolutional Neural Networks
- Chinese Character Recognition Dataset
- Training Our Models
- Hyperparameter Optimisation
- Transfer Learning

The convolution operation



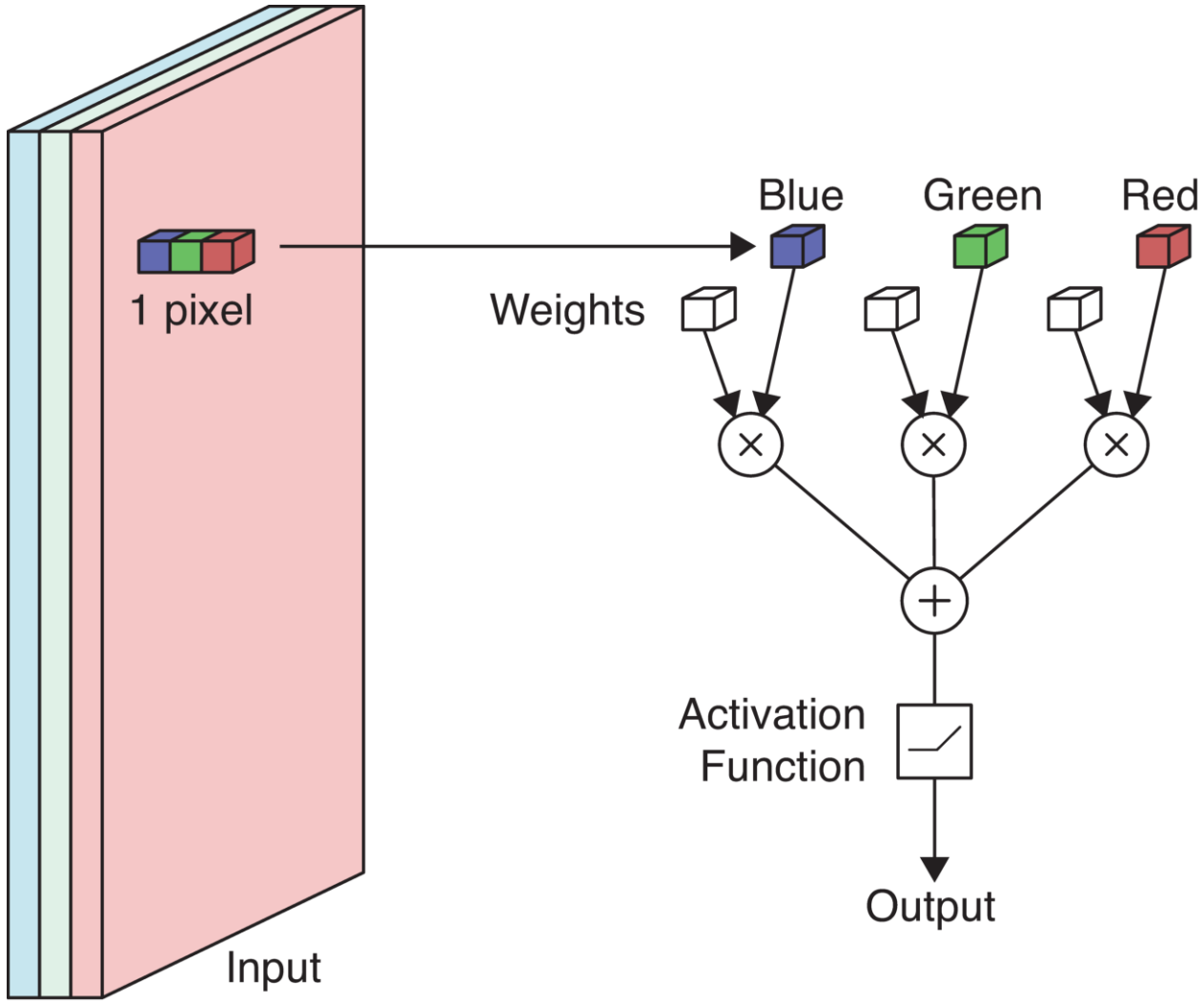
Scan the 3-channel input (colour image) with the neuron to produce a 1-channel output (grayscale image).

The output is produced by *sweeping* the neuron over the input. This is called **convolution**.

Aside: you'd have seen the convolution operation when calculating the density of $S = X_1 + X_2$ for i.i.d. $X_1, X_2 \sim f_X$ as $f_S(s) = \int f_X(x_1)\, f_X(s - x_1)\, \mathrm{d}x_1 = (f_X \star f_X)(s)$. This is why they're named "convolutional".

Source: Glassner (2021), Chapter 16.

The weights and biases



Applying a neuron to an image pixel.

Source: Glassner (2021), Chapter 16.

Example: Detecting yellow

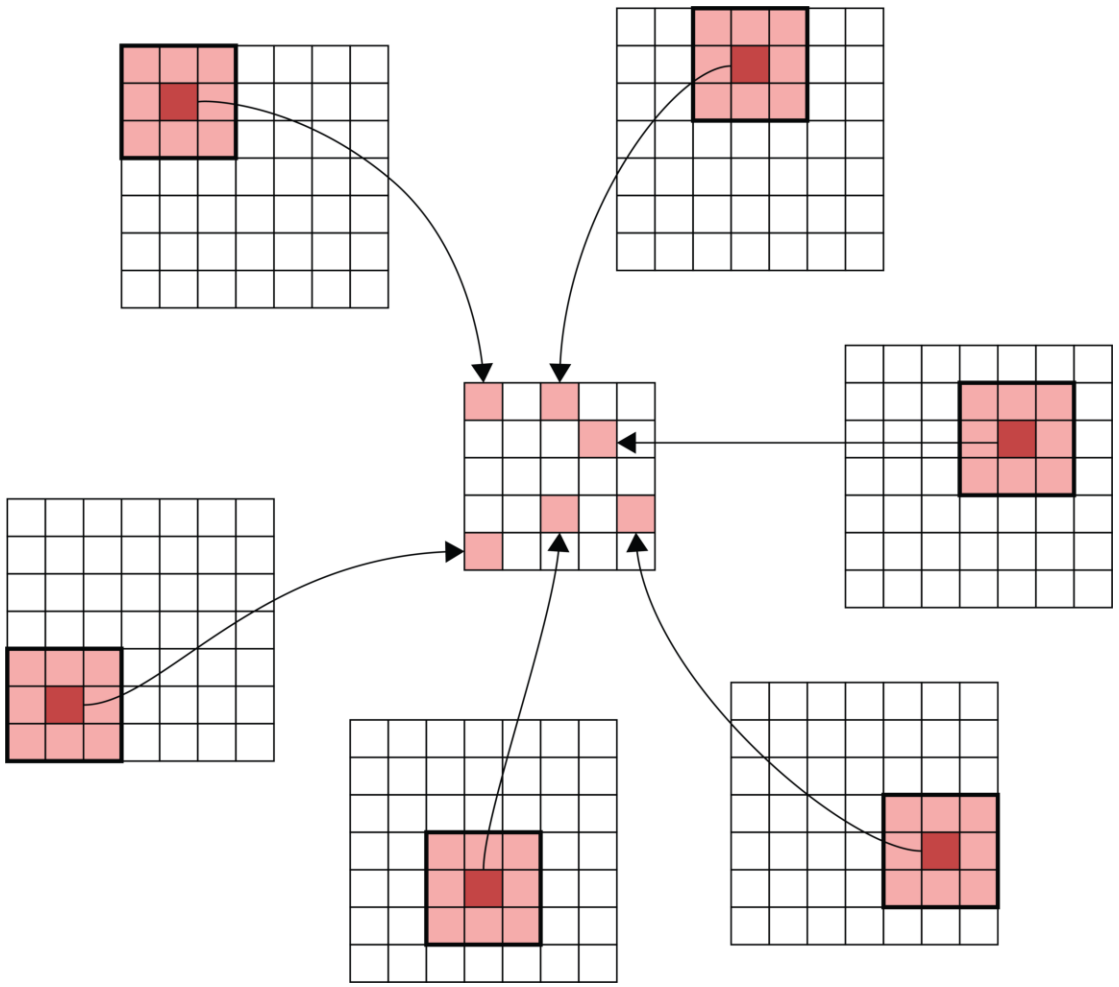
If red/green ↗ or blue ↘ then yellowness ↗. Set RGB weights to 1, 1, -1.

The more yellow the pixel in the colour image (left), the more white it is in the grayscale image.

This is about all we can detect by looking pixel-by-pixel. Typically we look at 3x3 or 5x5 blocks of pixels together.

Source: Glassner (2021), Chapter 16.

Filters

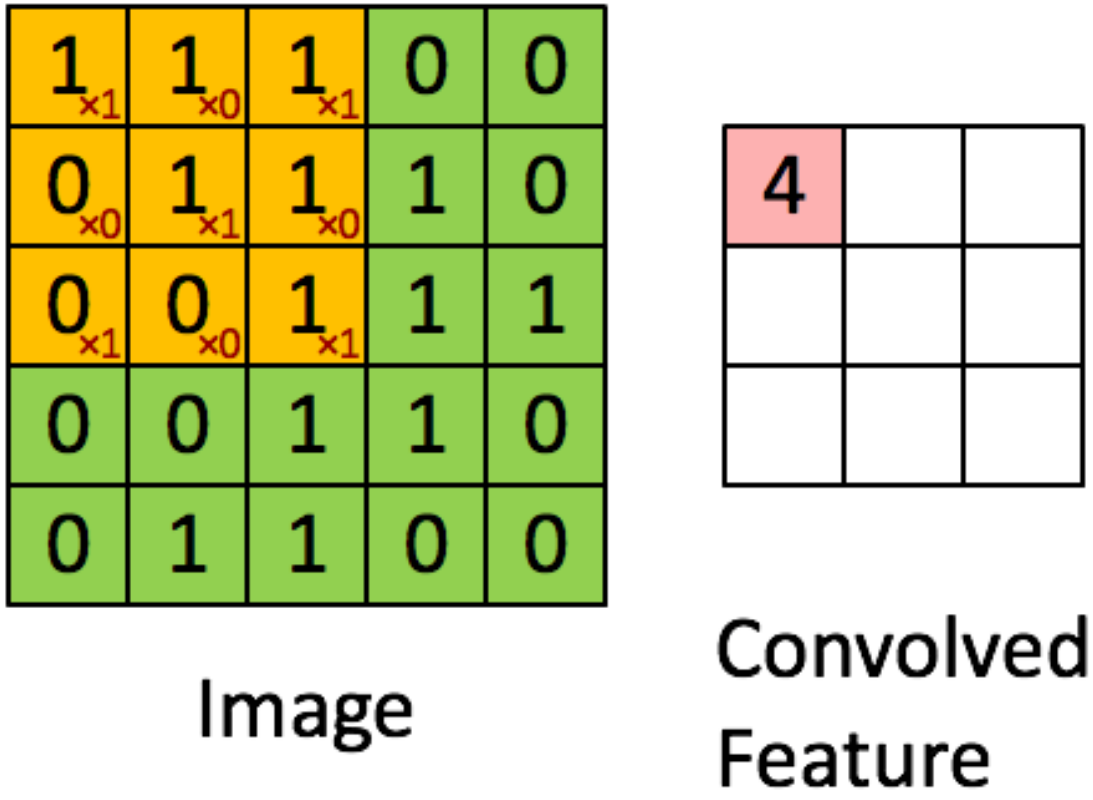


- The patch of input pixels a filter covers at one location is its *footprint*. Applied there, the filter turns that patch into a single output number — which is exactly what one neuron does.
- The *same* filter slides over every location instead of learning new weights per pixel. This *weight sharing* is why a convolution layer has so few parameters.

A *filter* (also called a *kernel*) is a small block of weights (plus a bias) which we sweep over the input in the convolution operation.

Source: Glassner (2021), Chapter 16.

Example filter

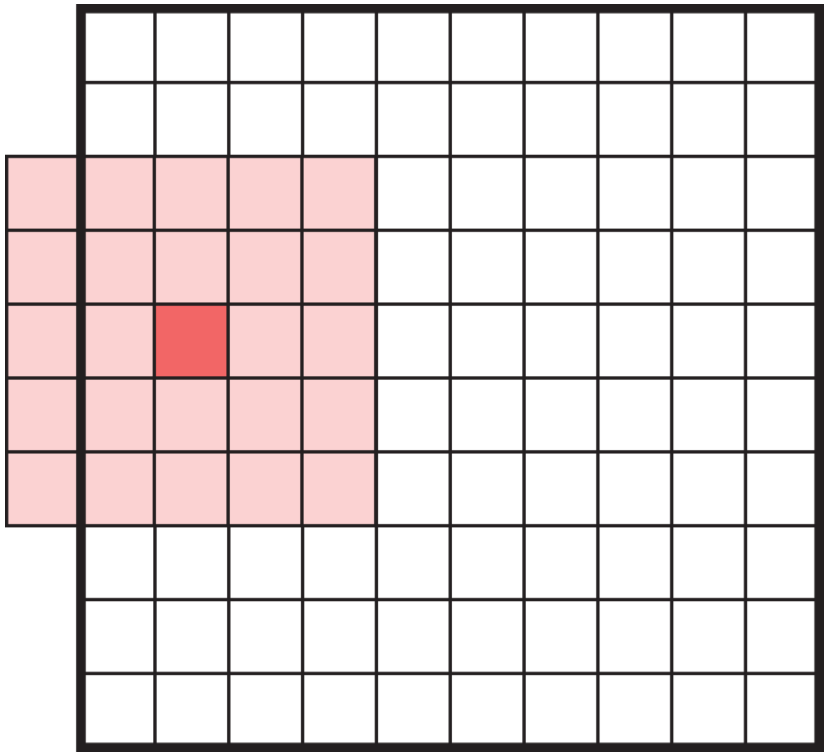


An example of filter convolution in action.

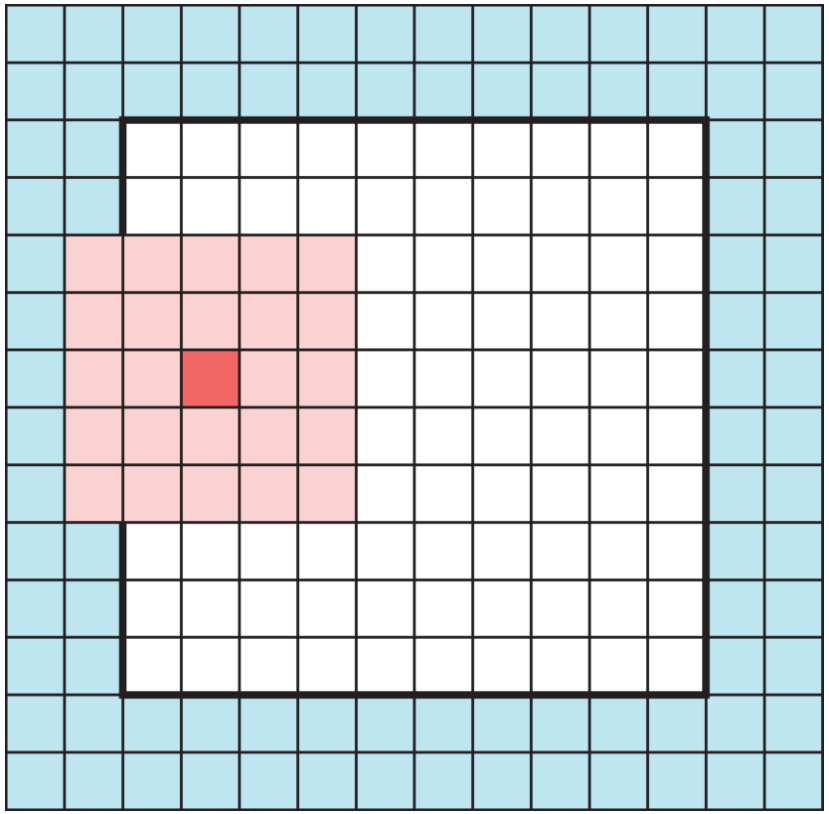
Take a look at https://setosa.io/ev/image-kernels/.

Source: Stanford's deep learning tutorial via Stack Exchange.

Padding



What happens when filters go off the edge of the input?



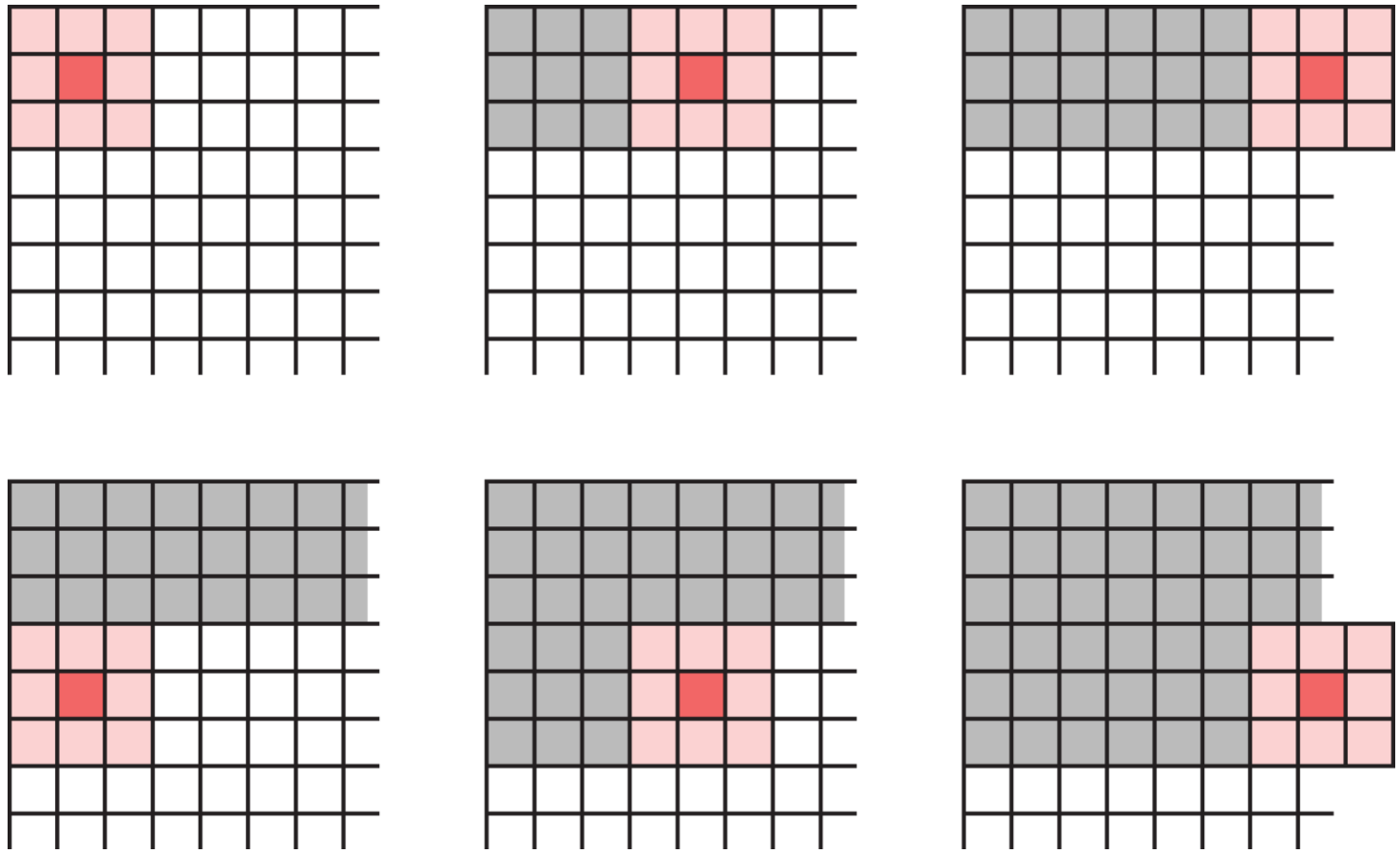
Add a border of zeros (“zero padding”) so the filter fits.

Add zeros around the input so the filter’s footprint doesn’t fall off the edge. This lets the output keep the same size as the input (`padding="same"`).

Source: Glassner (2021), Chapter 16.

Striding

We don't have to move the filter one pixel across/down at a time — a larger *stride* shrinks the output and saves computation.

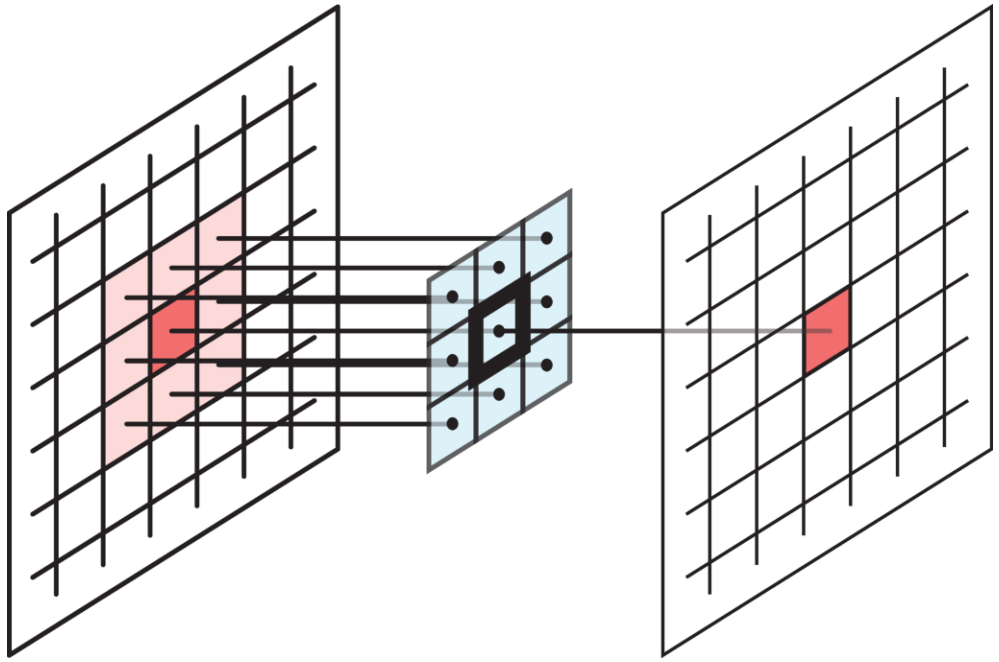


A 3x3 filter with stride 3 in both directions, so no input element is used more than once.

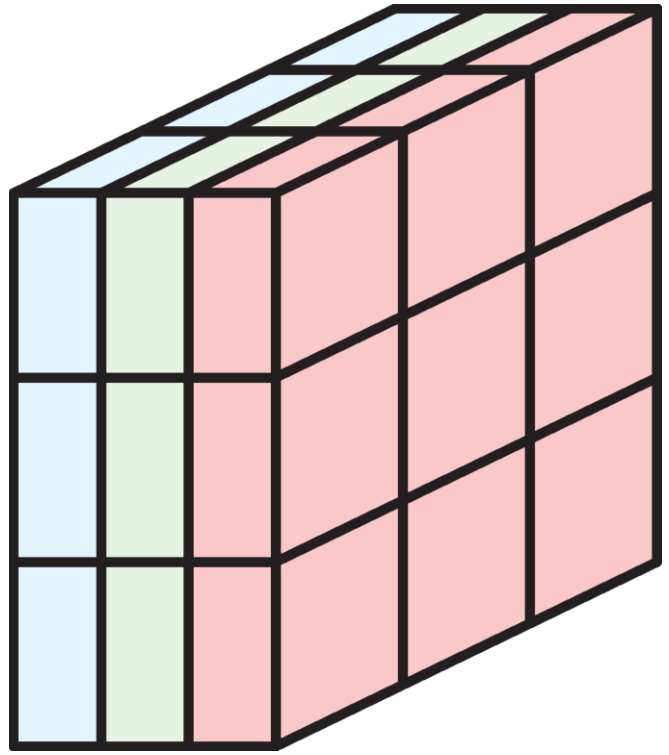
Source: Glassner (2021), Chapter 16.

Multidimensional convolution

A filter covers a *block* of pixels (e.g. a 3x3 filter has 9 weights), and must have the same number of channels as its input.



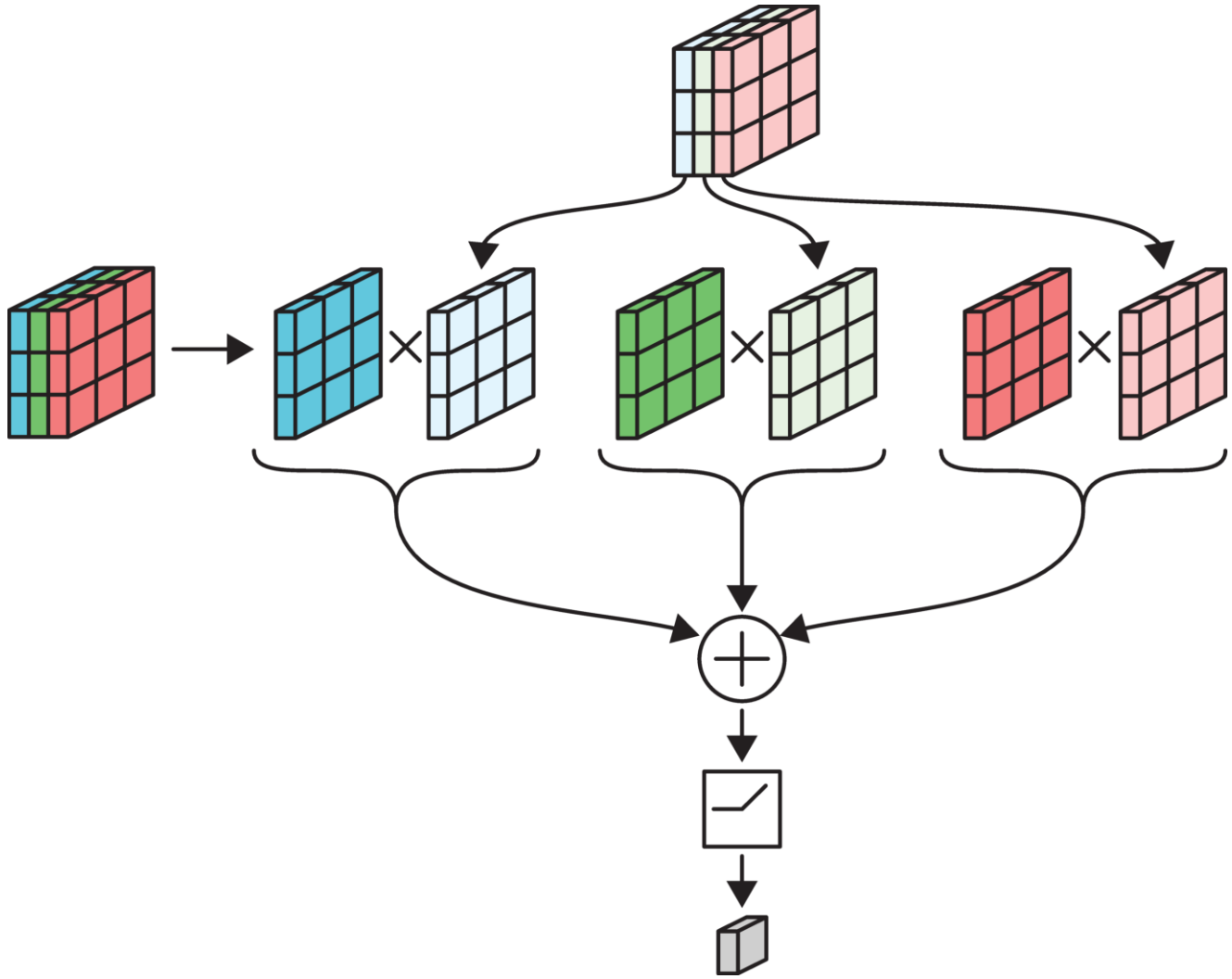
A 3x3 filter: 9 weights.



A 3x3 filter over 3 channels: 27 weights.

Source: Glassner (2021), Chapter 16.

Example: 3x3 filter over RGB input



Each channel is multiplied separately & then added together.

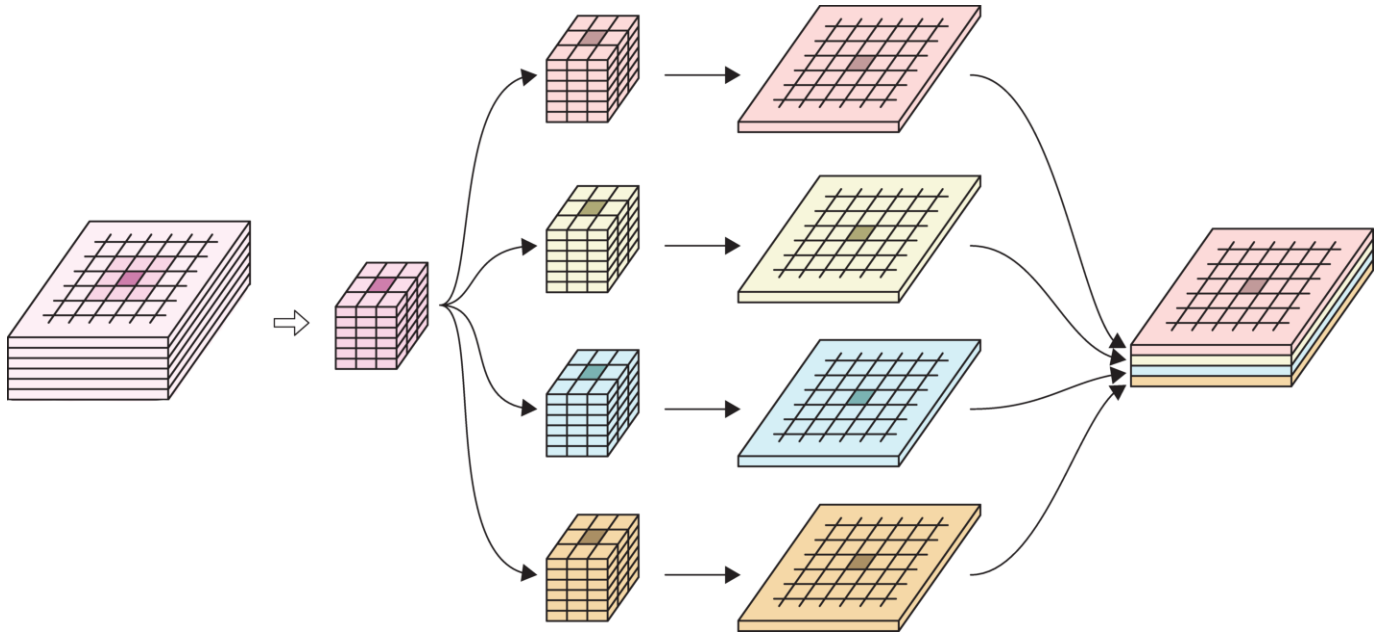
Source: Glassner (2021), Chapter 16.

Convolution layer

- Multiple filters are bundled together in one layer.
- The filters are applied *simultaneously* and *independently* to the input.
- Number of channels in the output will be the same as the number of filters.

In the image:

- 6-channel input tensor
- input pixels
- four 3x3 filters
- four output tensors
- final output tensor.



A layer with four 3x3(x6) filters.

Source: Glassner (2021), Chapter 16.

Specifying a convolutional layer

Need to choose:

- number of filters,
- their size/footprint (e.g. 3x3, 5x5, etc.),
- activation functions,
- padding & striding (optional).

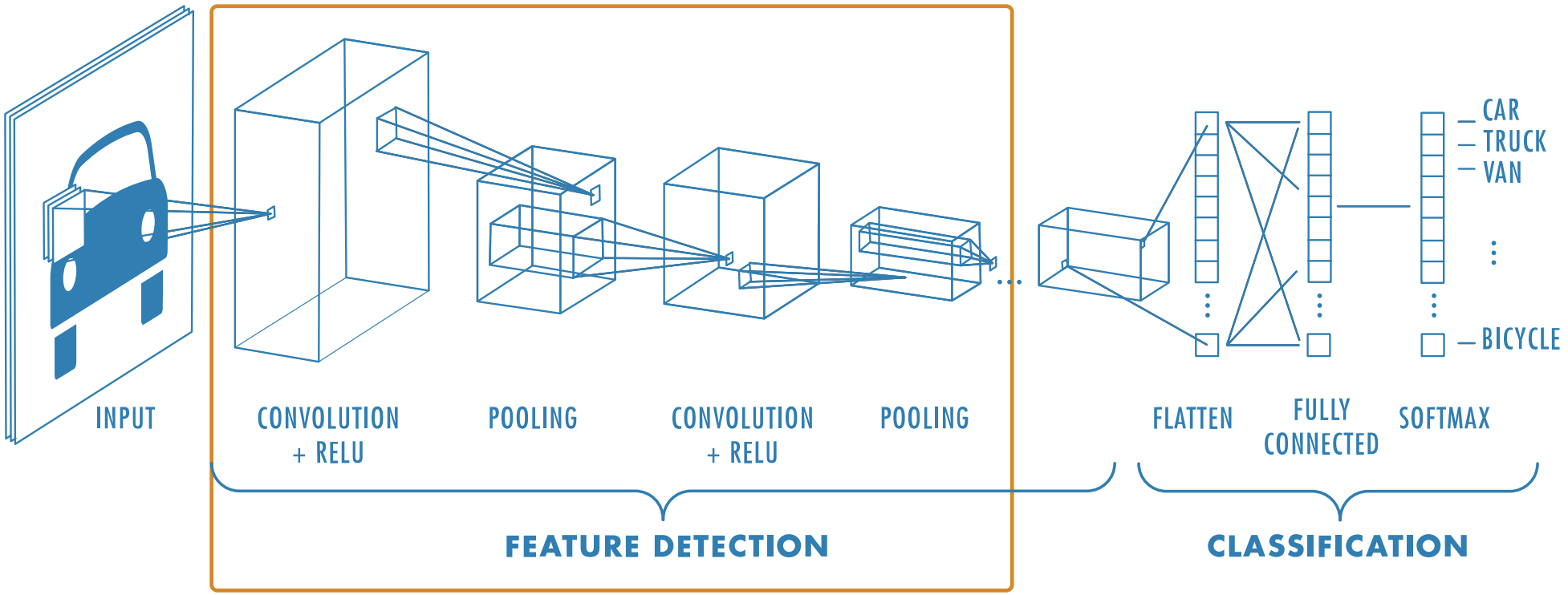
All the filter weights are learned during training.

Lecture Outline

- Introduction
- The Convolution Operation
- **Convolutional Neural Networks**
- Chinese Character Recognition Dataset
- Training Our Models
- Hyperparameter Optimisation
- Transfer Learning

Convolutional Neural Networks

A neural network that uses *convolution layers* is called a *convolutional neural network*.



A typical CNN structure.

Source: MathWorks, *Introducing Deep Learning with MATLAB*, Ebook.

Pooling

Pooling, or **downsampling**, is a technique to blur a tensor.

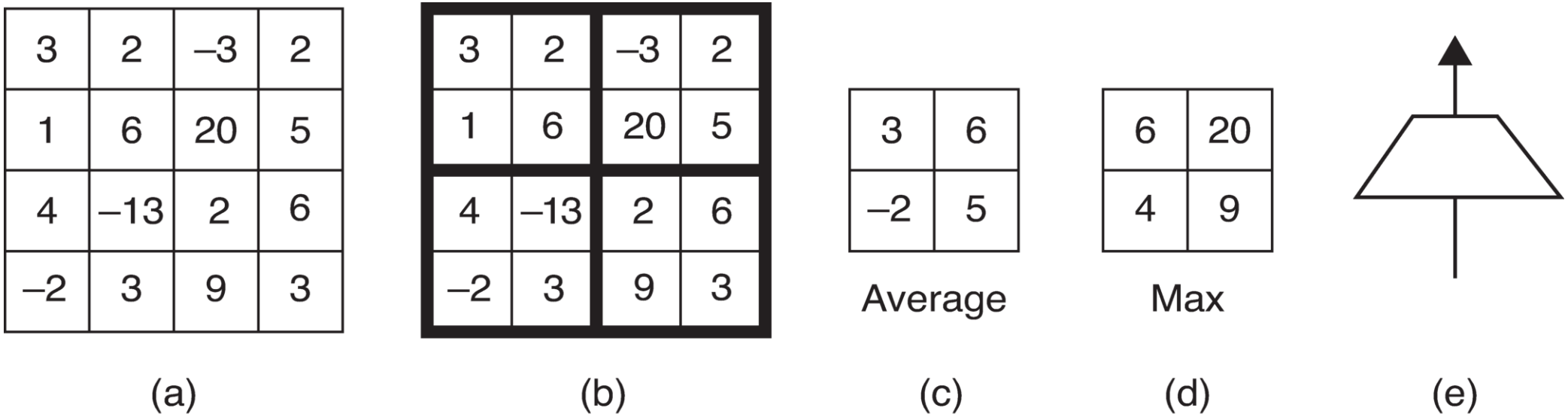
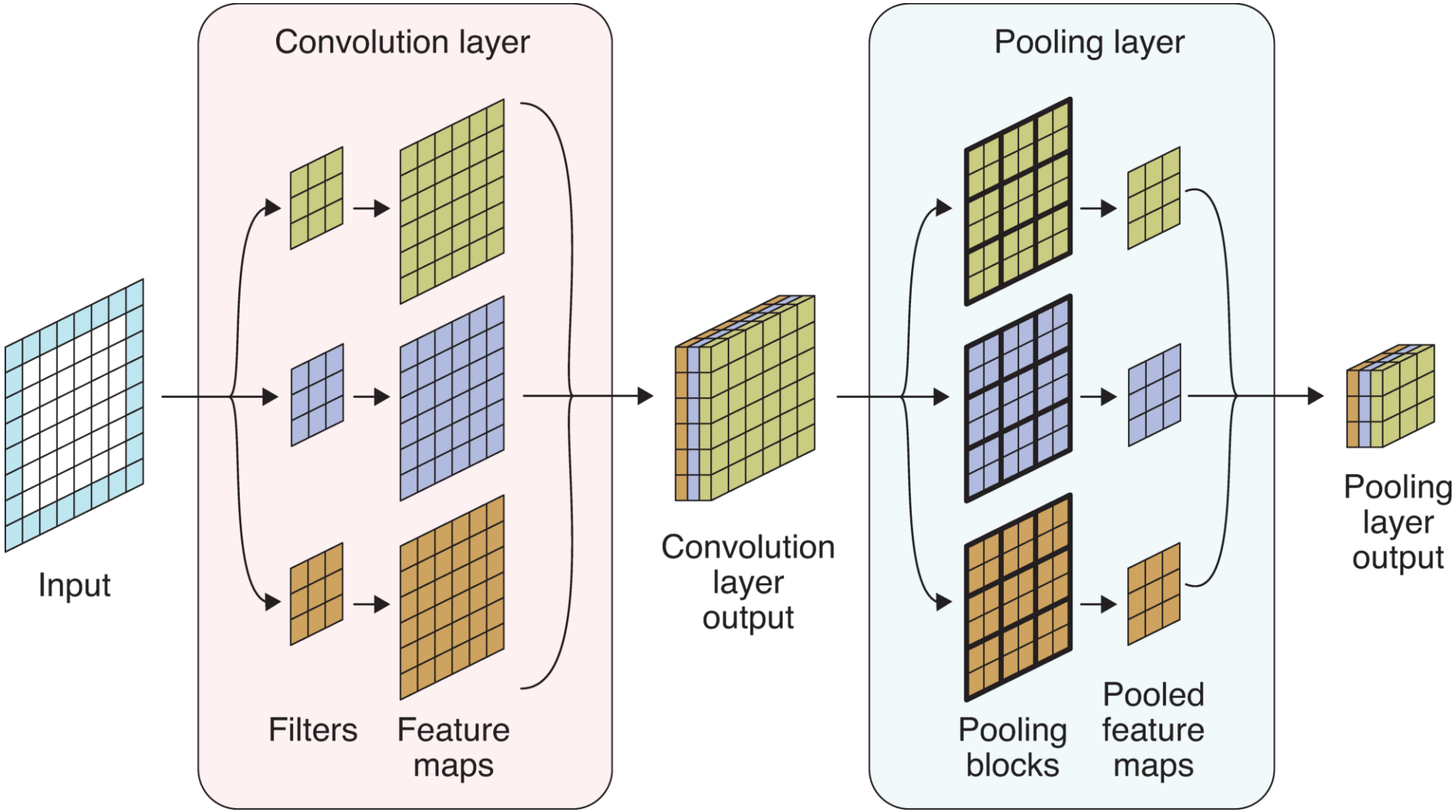


Illustration of pool operations.

(a): Input tensor (b): Subdivide input tensor into 2x2 blocks (c): Average pooling (d): Max pooling (e): Icon for a pooling layer

Source: Glassner (2021), Chapter 16.

Pooling for multiple channels



Pooling a multichannel input.

Source: Glassner (2021), Chapter 16.

MNIST Dataset

The MNIST dataset.

Source: Wikipedia, MNIST database.

LeNet-5 (1998)

Layer	Type	Channels	Size	Kernel size	Stride	Activation
In	Input	1	32×32	–	–	–
C1	Convolution	6	28×28	5×5	1	tanh
S2	Avg pooling	6	14×14	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S4	Avg pooling	16	5×5	2×2	2	tanh
C5	Convolution	120	1×1	5×5	1	tanh
F6	Fully connected	–	84	–	–	tanh
Out	Fully connected	–	10	–	–	RBF

Note

MNIST images are 28×28 pixels, and with zero-padding (for a 5×5 kernel) that becomes 32×32.

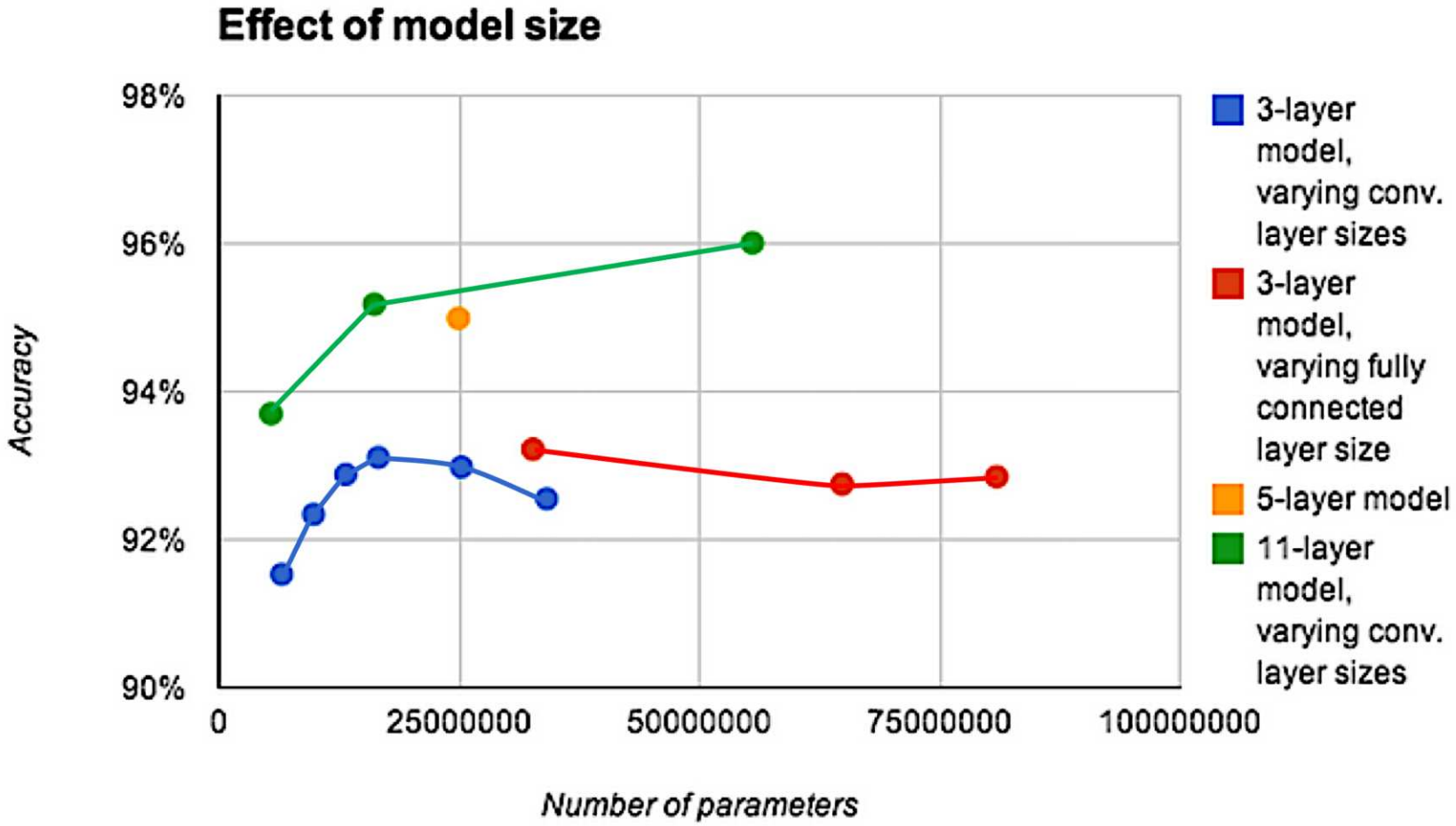
Source: Géron (2022), Chapter 14.

AlexNet (2012)

Layer	Type	Channels	Size	Kernel	Stride	Padding	Activation
In	Input	3	227×227	–	–	–	–
C1	Convolution	96	55×55	11×11	4	valid	ReLU
S2	Max pool	96	27×27	3×3	2	valid	–
C3	Convolution	256	27×27	5×5	1	same	ReLU
S4	Max pool	256	13×13	3×3	2	valid	–
C5	Convolution	384	13×13	3×3	1	same	ReLU
C6	Convolution	384	13×13	3×3	1	same	ReLU
C7	Convolution	256	13×13	3×3	1	same	ReLU
S8	Max pool	256	6×6	3×3	2	valid	–
F9	Fully conn.	–	4,096	–	–	–	ReLU
F10	Fully conn.	–	4,096	–	–	–	ReLU
Out	Fully conn.	–	1,000	–	–	–	Softmax

Winner of the ILSVRC 2012 challenge (top-five error 17%), developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton.

Depth can be important for image tasks



Deeper models aren't just better because they have more parameters.

Source: Adapted from Goodfellow et al. (2014).

Residual connection

Residual (skip) connections (He et al., 2016) let the network learn a correction to the identity, making it possible to train hundreds of layers.

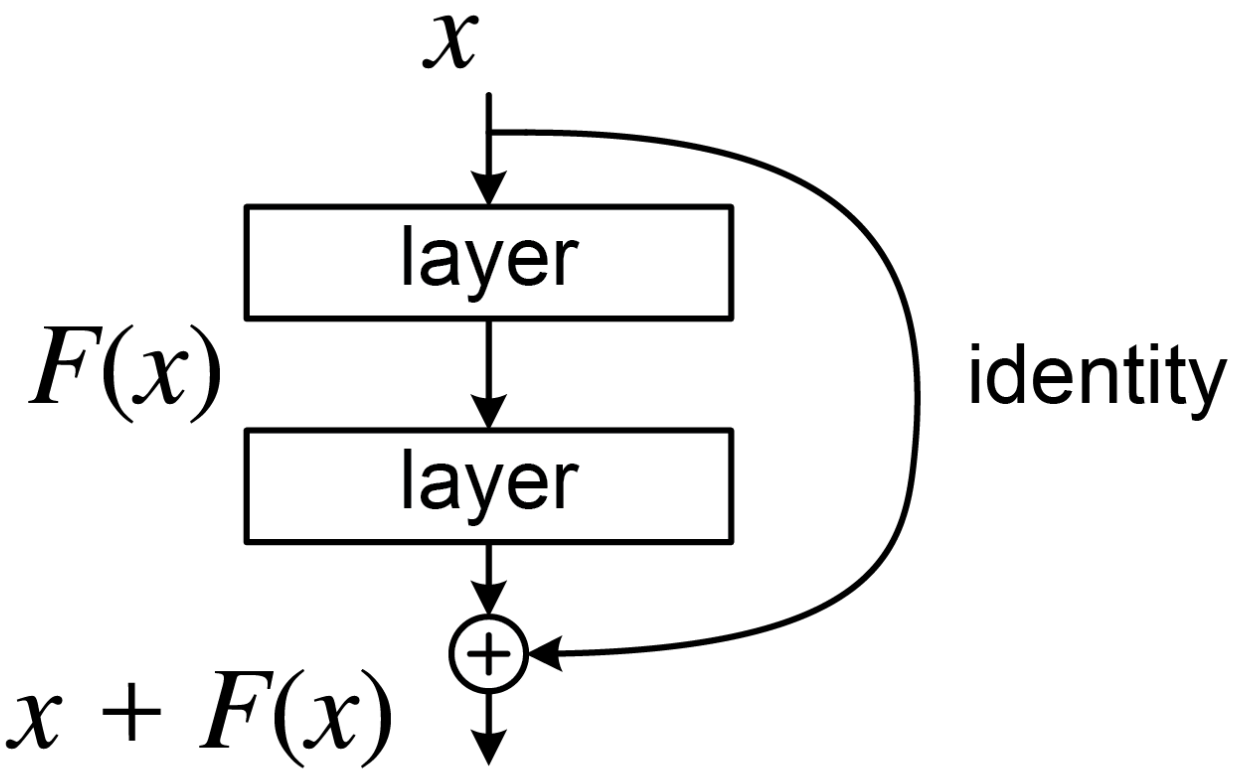


Illustration of a residual connection.

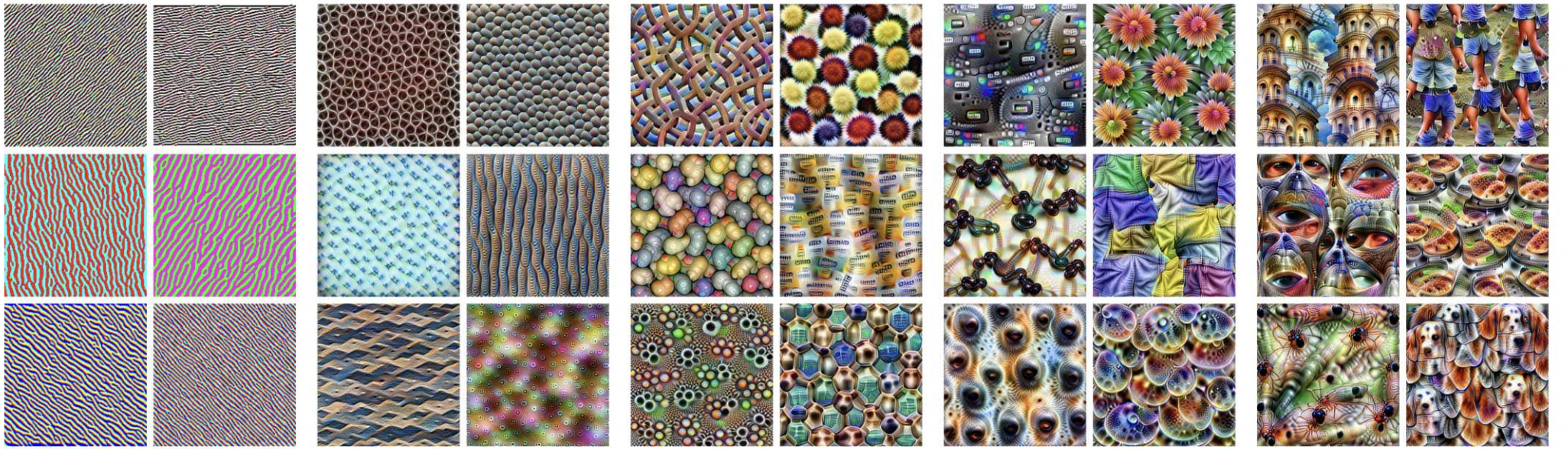
Source: Wikimedia

Image augmentation

One image becomes many training examples, using Keras' built-in augmentation layers.

Base image: Grace Hopper (ships with Matplotlib).

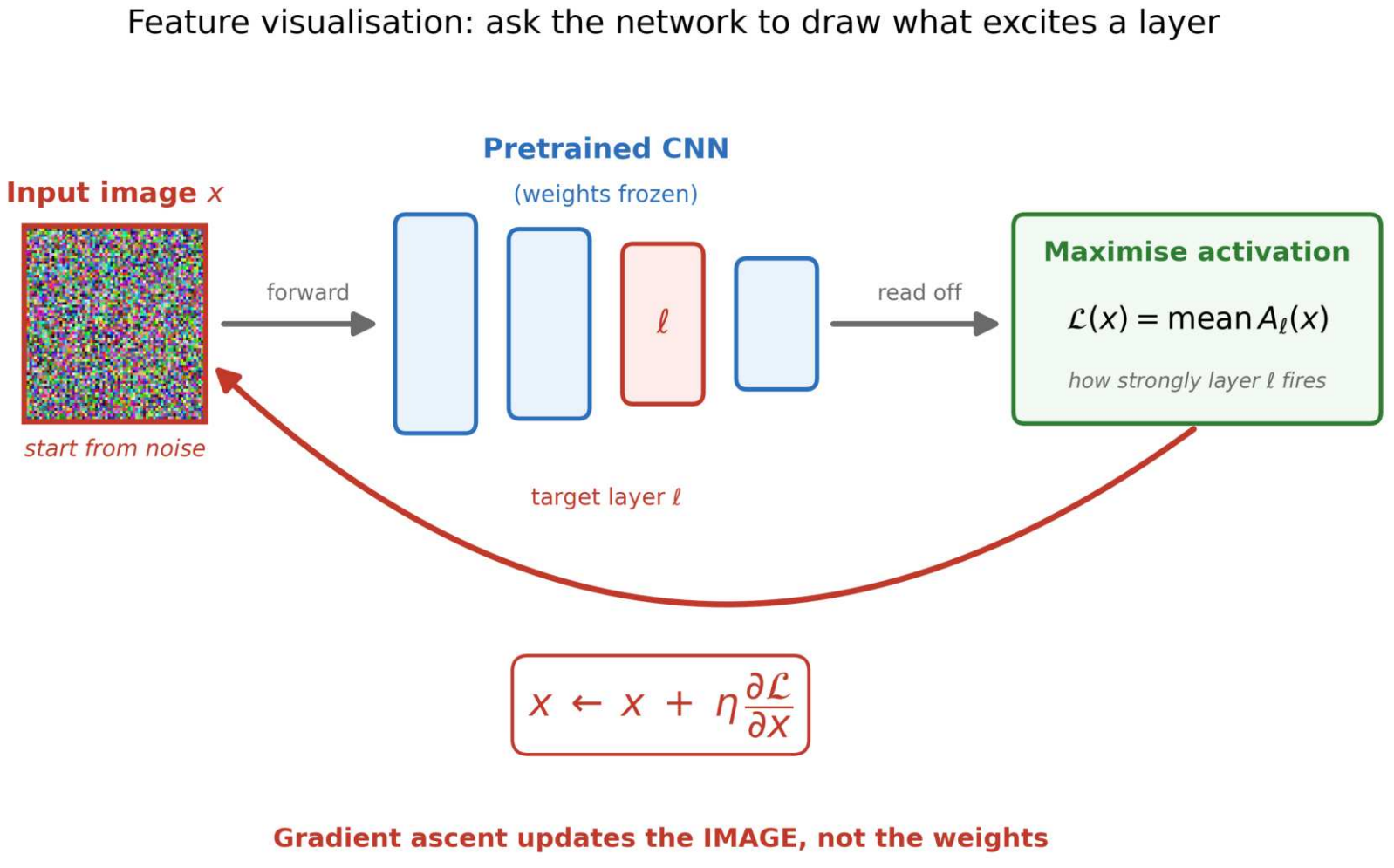
What do the CNN layers learn?



Early layers learn simple patterns; deep layers, complex ones.

Source: Distill article, Feature Visualization.

How does that work?



Start from a noise image and nudge its *pixels* until a chosen layer fires hardest.

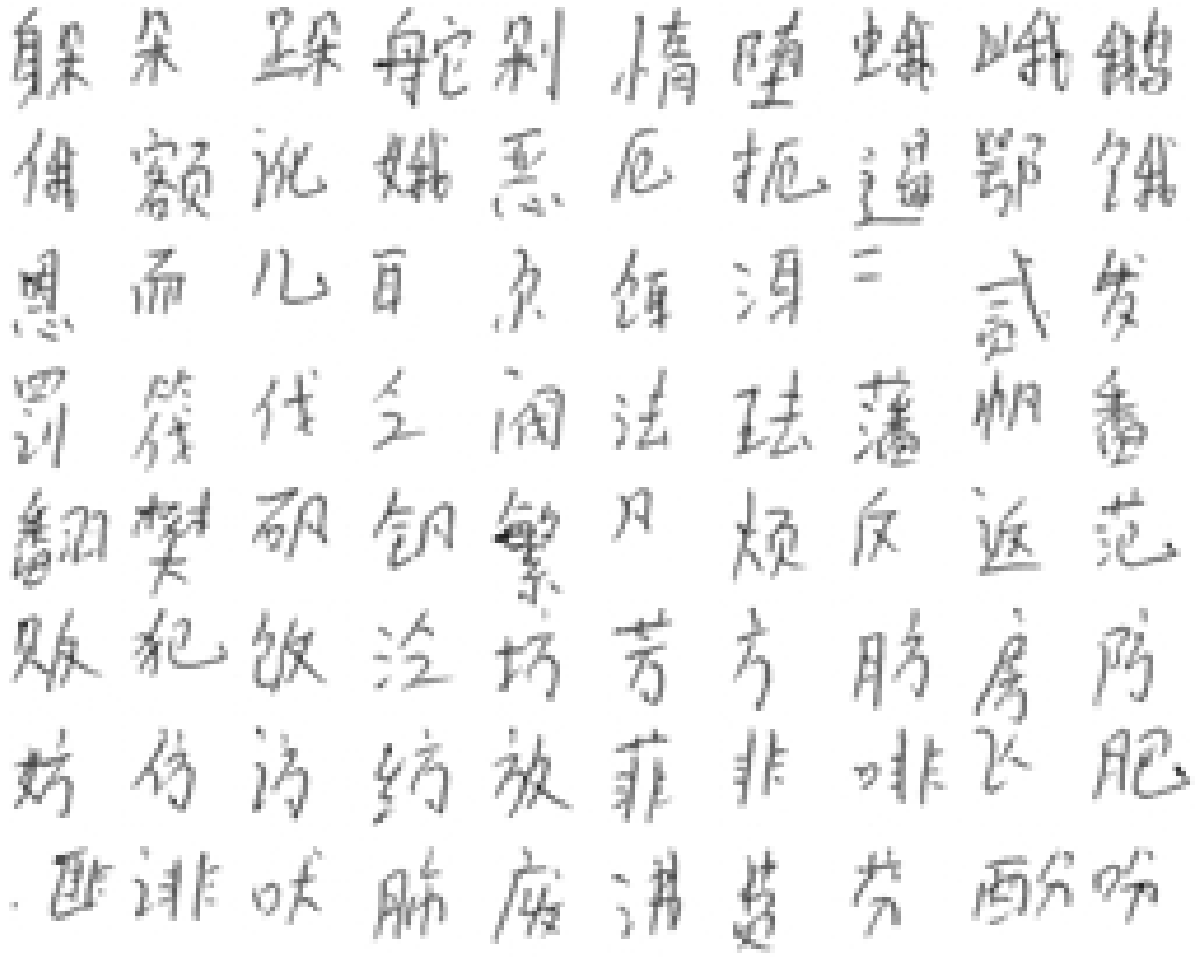
Method from the Distill article, Feature Visualization.

Lecture Outline

- Introduction
- The Convolution Operation
- Convolutional Neural Networks
- **Chinese Character Recognition Dataset**
- Training Our Models
- Hyperparameter Optimisation
- Transfer Learning

CASIA Chinese handwriting database

Dataset source: Institute of Automation of Chinese Academy of Sciences (CASIA)



A 13 GB dataset of 3,999,571 handwritten characters.

Source: Liu et al. (2011). Dataset available here.

Inspect a subset of characters

Pulling out 55 characters to experiment with.

Inspect directory structure

```
1 DisplayTree("CASIA-Dataset")
```

```
CASIA-Dataset/
├── Test/
│   ├──   /
│   │   ├── 1.png
│   │   ├── 10.png
│   │   ├── 100.png
│   │   ├── 101.png
│   │   ├── 102.png
│   │   ├── 103.png
│   │   ├── 104.png
│   │   ├── 105.png
│   │   ├── 106.png
...
│   │   ├── 97.png
│   │   ├── 98.png
│   │   └── 99.png
```

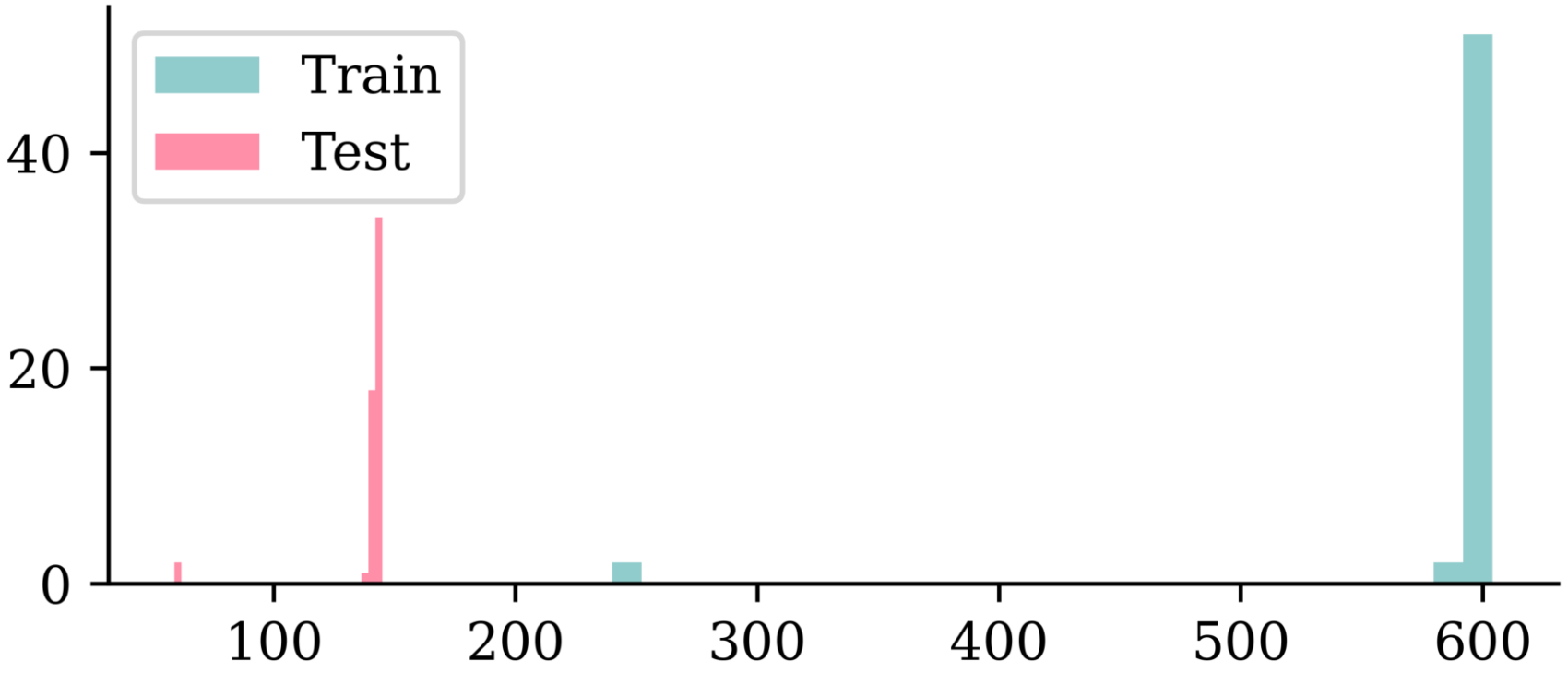
Count number of images for each character

```python
def count_images_in_folders(root_folder):
    counts = {}
    for folder in root_folder.glob("*/"):
        counts[folder.name] = len(list(folder.glob("*.png")))
    return counts

train_counts = count_images_in_folders(Path("CASIA-Dataset/Train"))
test_counts = count_images_in_folders(Path("CASIA-Dataset/Test"))

print(train_counts)
print(test_counts)
```

```
{'  ': 584, '  ': 597, '  ': 597, '  ': 240, '  ': 240, '  ': 596, '  ': 596, '  ': 598, '  ':
600, '  ': 597, '  ': 604, '  ': 599, '  ': 603, '  ': 598, '  ': 604, '  ': 593, '  ': 599,
'  ': 602, '  ': 602, '  ': 600, '  ': 597, '  ': 598, '  ': 601, '  ': 598, '  ': 591, '  ':
597, '  ': 598, '  ': 598, '  ': 601, '  ': 597, '  ': 601, '  ': 598, '  ': 597, '  ': 600,
'  ': 601, '  ': 597, '  ': 596, '  ': 598, '  ': 597, '  ': 603, '  ': 598, '  ': 595, '  ':
602, '  ': 604, '  ': 595, '  ': 597, '  ': 601, '  ': 598, '  ': 601, '  ': 600, '  ': 600,
'  ': 602, '  ': 602, '  ': 599, '  ': 599}
{'  ': 138, '  ': 143, '  ': 144, '  ': 60, '  ': 59, '  ': 144, '  ': 143, '  ': 144, '  ': 142,
'  ': 144, '  ': 143, '  ': 141, '  ': 144, '  ': 143, '  ': 144, '  ': 142, '  ': 144, '  ':
143, '  ': 142, '  ': 141, '  ': 143, '  ': 141, '  ': 140, '  ': 142, '  ': 144, '  ': 141,
'  ': 143, '  ': 143, '  ': 142, '  ': 144, '  ': 143, '  ': 142, '  ': 143, '  ': 142, '  ':
141, '  ': 144, '  ': 143, '  ': 144, '  ': 143, '  ': 144, '  ': 143, '  ': 142, '  ': 144,
'  ': 141, '  ': 144, '  ': 143, '  ': 143, '  ': 144, '  ': 145, '  ': 143, '  ': 144, '  ':
143, '  ': 142, '  ': 141, '  ': 142}
```

Number of images for each character

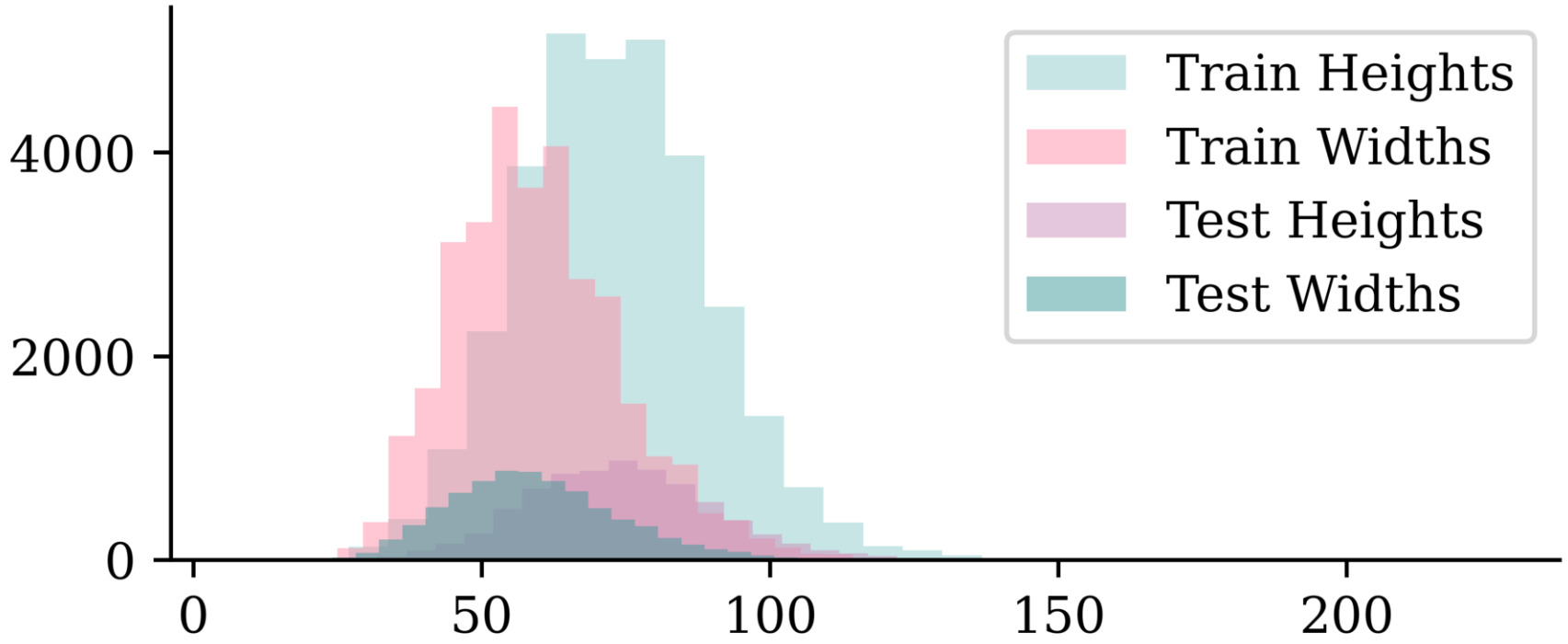
```
1 plt.hist(train_counts.values(), bins=30, label="Train")
2 plt.hist(test_counts.values(), bins=30, label="Test")
3 plt.legend();
```



It differs, but basically ~600 training and ~140 test images per character. A couple of characters have a lot less of both though.

Checking the dimensions

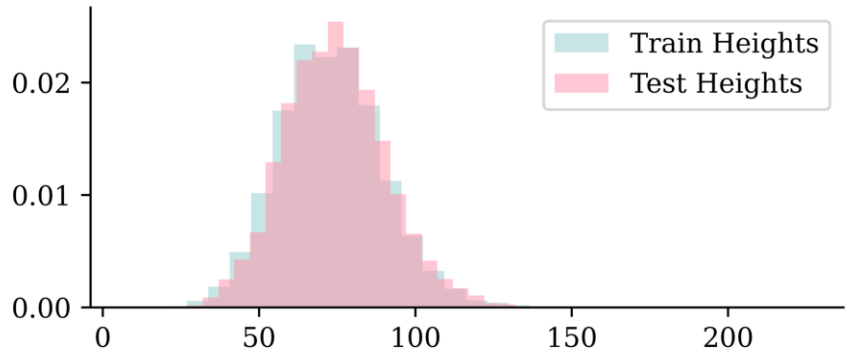
```
1 plt.hist(train_heights, bins=30, alpha=0.5, label="Train Heights")
2 plt.hist(train_widths, bins=30, alpha=0.5, label="Train Widths")
3 plt.hist(test_heights, bins=30, alpha=0.5, label="Test Heights")
4 plt.hist(test_widths, bins=30, alpha=0.5, label="Test Widths")
5 plt.legend();
```



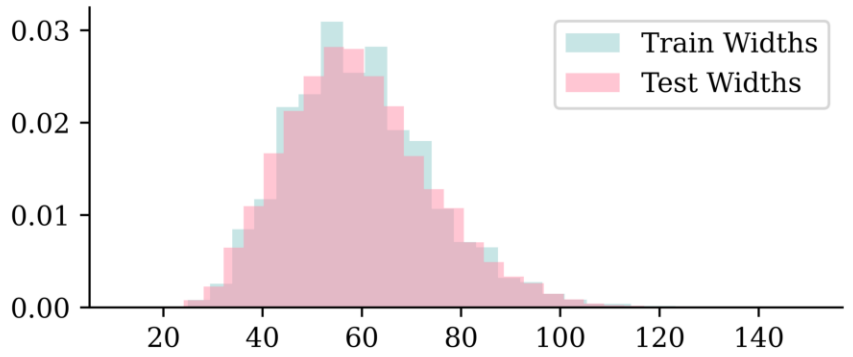
Checking the dimensions II

Using `density=True` removes the count imbalance, so we can compare the *shapes* of the distributions.

```
1 plt.hist(train_heights, bins=30, alpha=0.
2 plt.hist(test_heights, bins=30, alpha=0.5
3 plt.legend();
```




```
1 plt.hist(train_widths, bins=30, alpha=0.5
2 plt.hist(test_widths, bins=30, alpha=0.5,
3 plt.legend();
```



- The images are taller than they are wide.
- The distribution of dimensions are pretty similar between training and test sets.

Keras image dataset loading

Normally we'd used `keras.utils.image_dataset_from_directory` but the Chinese characters breaks it on Windows. I made an image loading function just for this demo.

```
data_dir = Path("CASIA-Dataset")
img_height, img_width = 80, 60  # Target image size

# Load 'training' and test datasets
X_main, y_main, class_names = load_images_from_directory(data_dir / "Train", img_height,
X_test, y_test, _ = load_images_from_directory(data_dir / "Test", img_height, img_width)

# Verify dataset shape
print(f"Train: X={X_main.shape}, y={y_main.shape}")
print(f"Test: X={X_test.shape}, y={y_test.shape}")
print("Class Names:", class_names)
```

```
Train: X=(32206, 80, 60, 1), y=(32206,)
Test: X=(7684, 80, 60, 1), y=(7684,)
Class Names: ['  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ',
'  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ',
'  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ',
'  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ', '  ']
```

Some setup

```
X_train, X_val, y_train, y_val = train_test_split(X_main, y_main, test_size=0.2,
    random_state=123)
print(X_train.shape, y_train.shape, X_val.shape, y_val.shape, X_test.shape, y_test.shape)
```

```
(25764, 80, 60, 1) (25764,) (6442, 80, 60, 1) (6442,) (7684, 80, 60, 1) (7684,)
```

```
CHINESE_FONT = fm.FontProperties(fname="STHeitiTC-Medium-01.ttf")

def plot_mandarin_characters(X, y, class_names, n=5, title_font=CHINESE_FONT):
    # Plot the first n images in X
    plt.figure(figsize=(10, 4))
    for i in range(n):
        plt.subplot(1, n, i + 1)
        plt.imshow(X[i], cmap="gray")
        plt.title(class_names[y[i]], fontproperties=title_font)
        plt.axis("off")
```

```
class_names[:5]
```

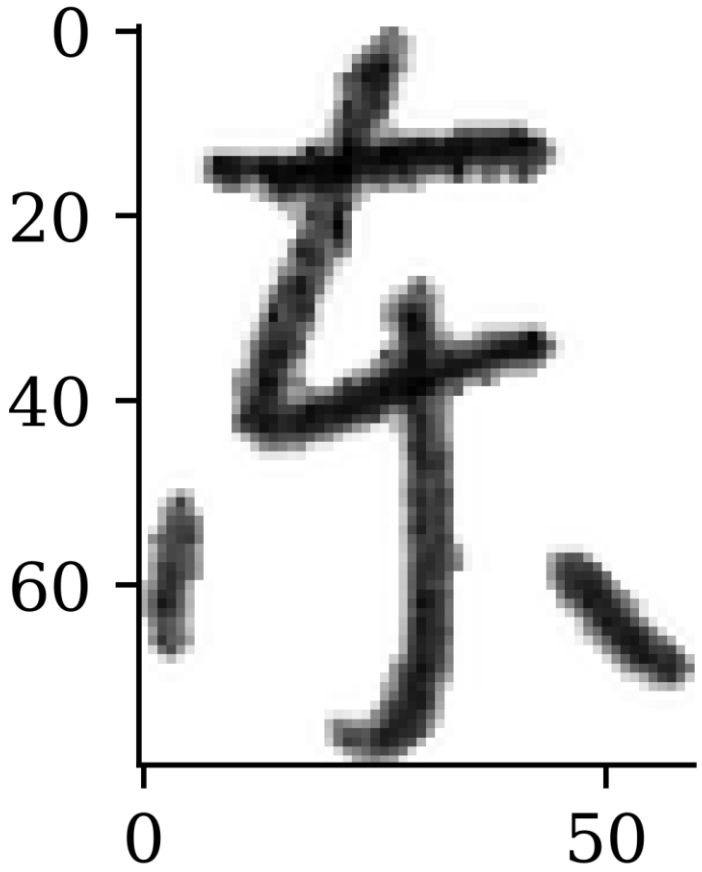
```
['  ', '  ', '  ', '  ', '  ']
```

```
X_dong = X_train[y_train == 0]; y_dong = y_train[y_train == 0]
X_ren = X_train[y_train == 2]; y_ren = y_train[y_train == 2]
```

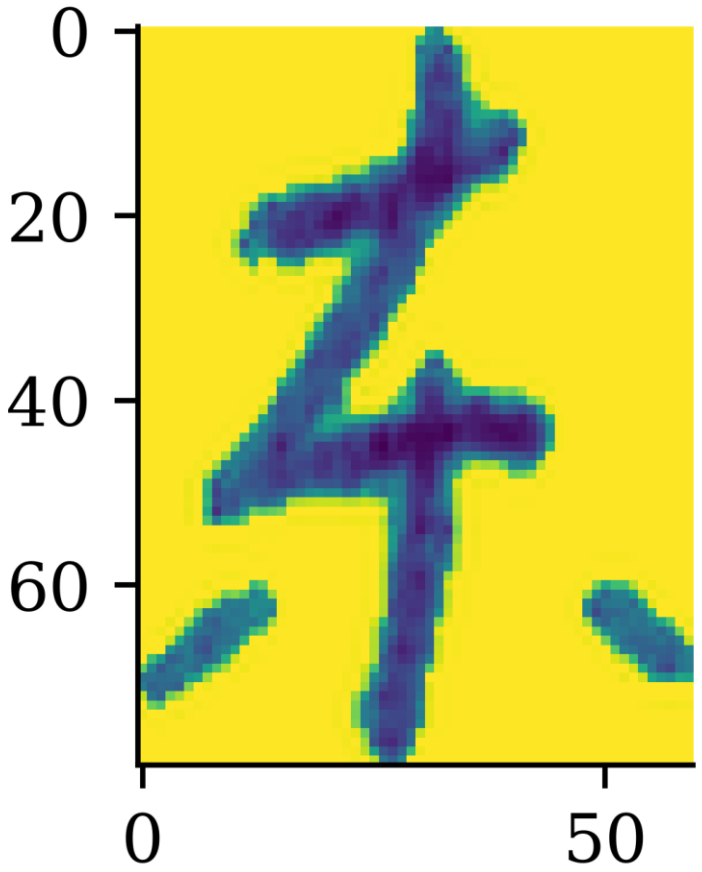
Plotting some training characters

Without the colourmap..

```
1 dong = X_test[y_test == 0][0]
2 plt.imshow(dong, cmap="gray");
```



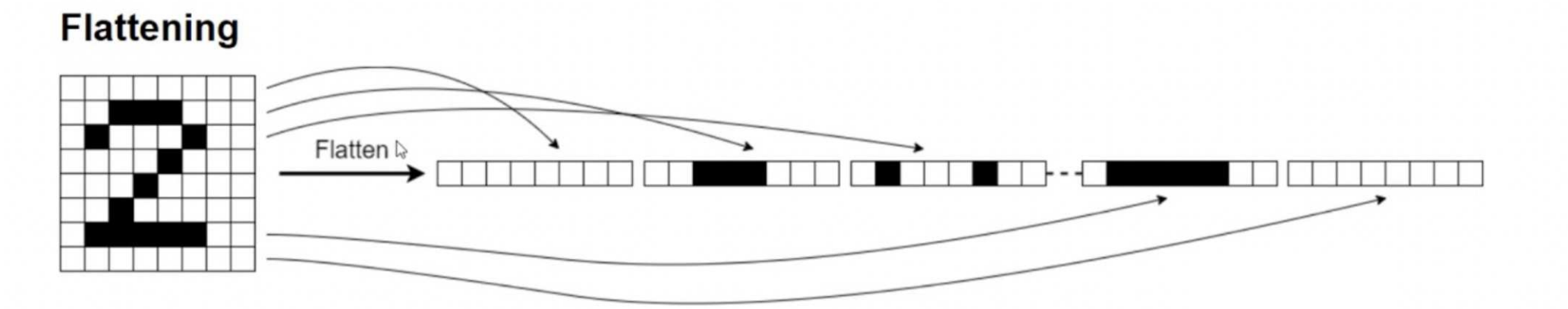

```
1 dong = X_test[y_test == 0][1]
2 plt.imshow(dong);
```



Lecture Outline

- Introduction
- The Convolution Operation
- Convolutional Neural Networks
- Chinese Character Recognition Dataset
- **Training Our Models**
- Hyperparameter Optimisation
- Transfer Learning

Make simple baseline (multinomial) logistic regression



Basically pretend it's not an image

```
num_classes = np.unique(y_train).shape[0]
random.seed(123)
model = Sequential([
  Input((img_height, img_width, 1)), Flatten(), Rescaling(1./255),
  Dense(num_classes, activation="softmax")
])
```

Tip

The `Rescaling` layer will rescale the intensities to [0, 1].

Inspecting the model

```
1 model.summary()
```

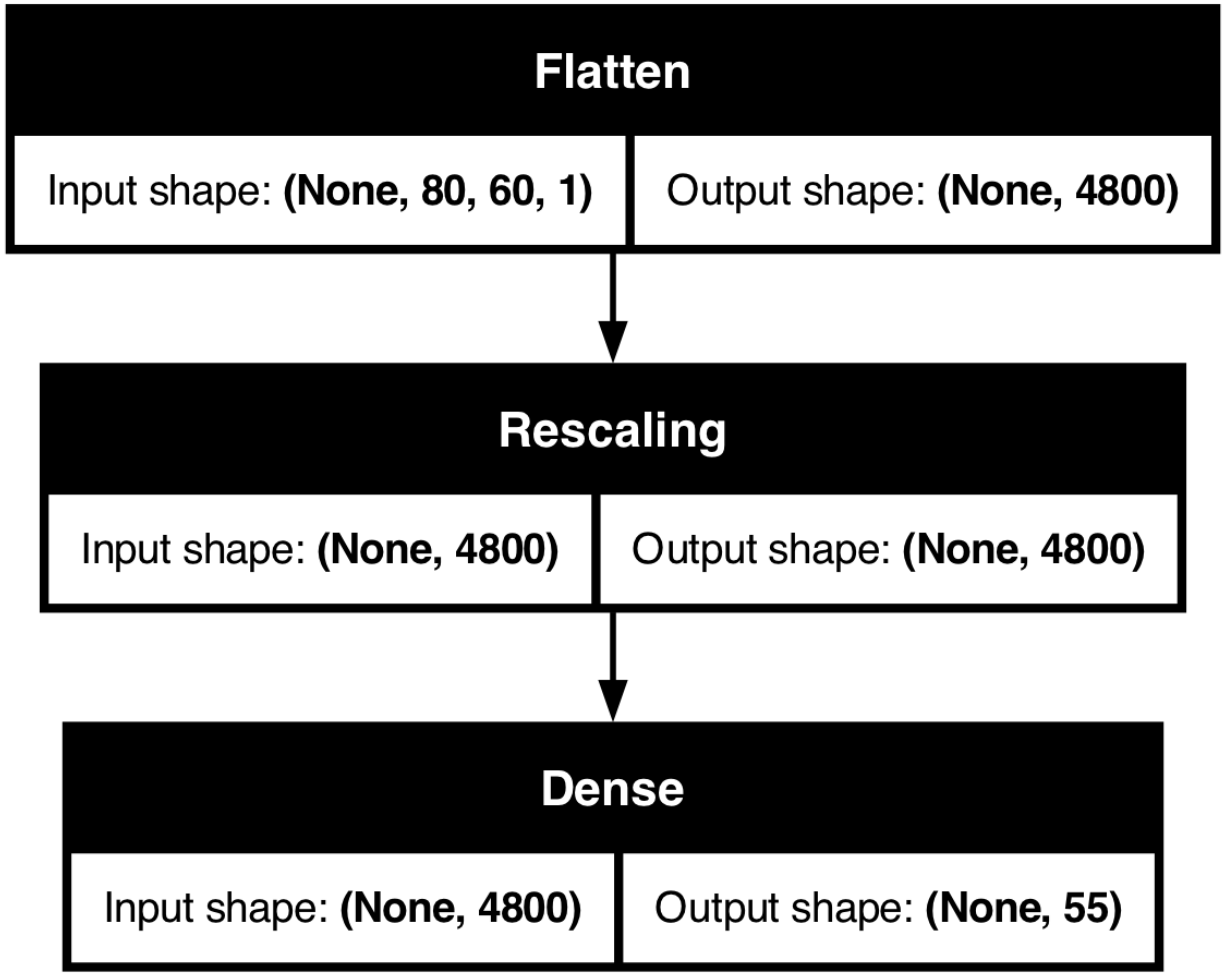
Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 4800)	0
rescaling (Rescaling)	(None, 4800)	0
dense (Dense)	(None, 55)	264,055

Total params: 264,055 (1.01 MB)
Trainable params: 264,055 (1.01 MB)
Non-trainable params: 0 (0.00 B)

Plot the model

```
1 plot_model(model, show_shapes=True)
```

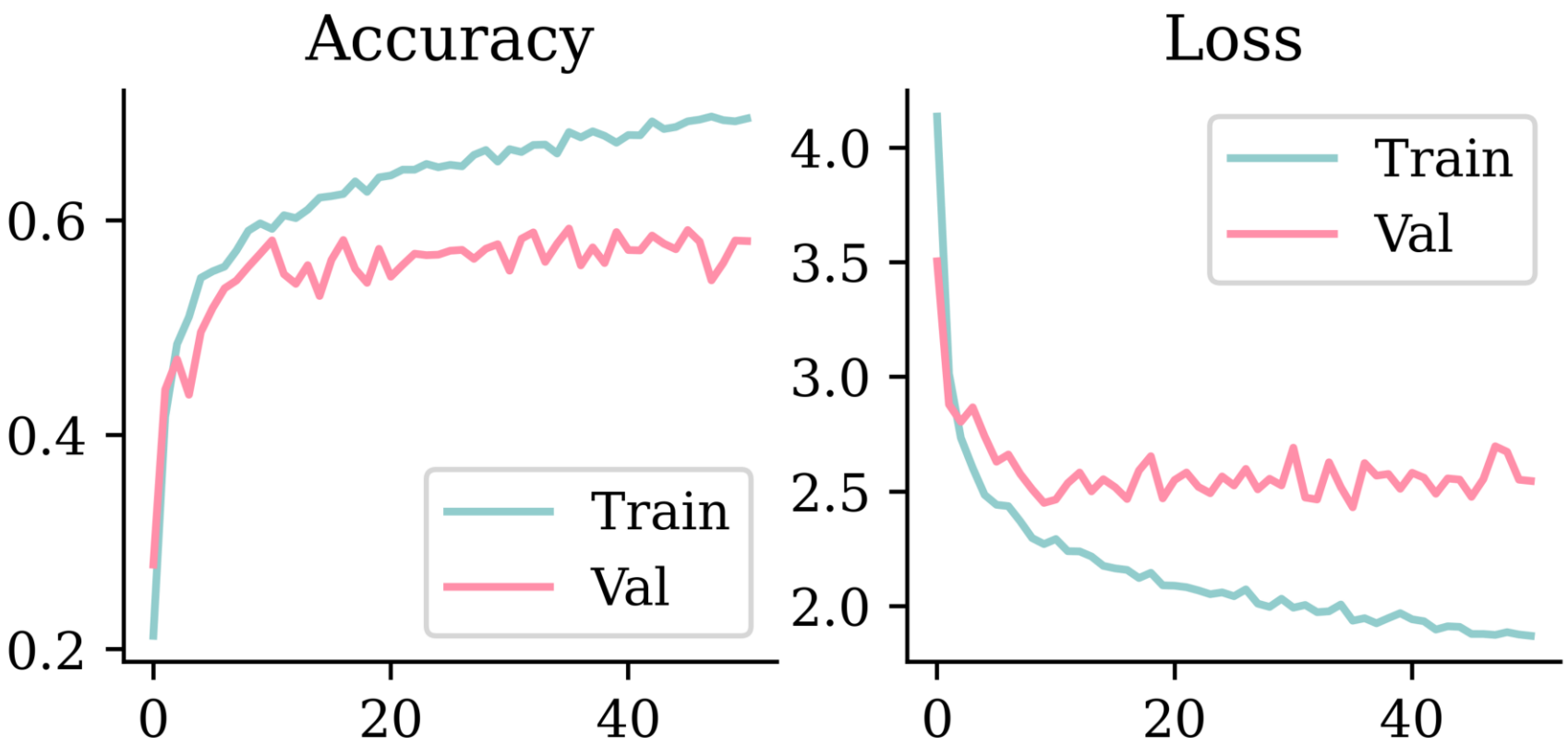


Fitting the model

```
loss = keras.losses.SparseCategoricalCrossentropy()
topk = keras.metrics.SparseTopKCategoricalAccuracy(k=5)
model.compile(optimizer='adam', loss=loss, metrics=['accuracy', topk])

es = EarlyStopping(patience=15, restore_best_weights=True,
    monitor="val_accuracy", verbose=2)
hist = model.fit(X_train, y_train, validation_data=(X_val, y_val),
    epochs=100, batch_size=128, callbacks=[es], verbose=0)
history = hist.history
```

Plot the loss/accuracy curves

```
1 plot_history(history)
```



Look at the metrics

```
print(model.evaluate(X_train, y_train, verbose=0))
print(model.evaluate(X_val, y_val, verbose=0))
```

```
[1.8805104494094849, 0.6948843598365784, 0.8843735456466675]
[2.4310331344604449, 0.592362642288208, 0.8346786499023438]
```

```
loss_value, accuracy, top5_accuracy = model.evaluate(X_val, y_val, verbose=0)
print(f"Validation Loss: {loss_value:.4f}")
print(f"Validation Accuracy: {accuracy:.4f}")
print(f"Validation Top 5 Accuracy: {top5_accuracy:.4f}")
```

```
Validation Loss: 2.4310
Validation Accuracy: 0.5924
Validation Top 5 Accuracy: 0.8347
```

Make a CNN

```
random.seed(123)

model = Sequential([
  Input((img_height, img_width, 1)),
  Rescaling(1./255),
  Conv2D(16, 3, padding="same", activation="relu", name="conv1"),
  MaxPooling2D(name="pool1"),
  Conv2D(32, 3, padding="same", activation="relu", name="conv2"),
  MaxPooling2D(name="pool2"),
  Conv2D(64, 3, padding="same", activation="relu", name="conv3"),
  MaxPooling2D(name="pool3", pool_size=(4, 4)),
  Flatten(),
  Dense(64, activation="relu"),
  Dense(num_classes)
])
```

Architecture inspired by https://www.tensorflow.org/tutorials/images/classification.

Inspect the model

```
1 model.summary()
```

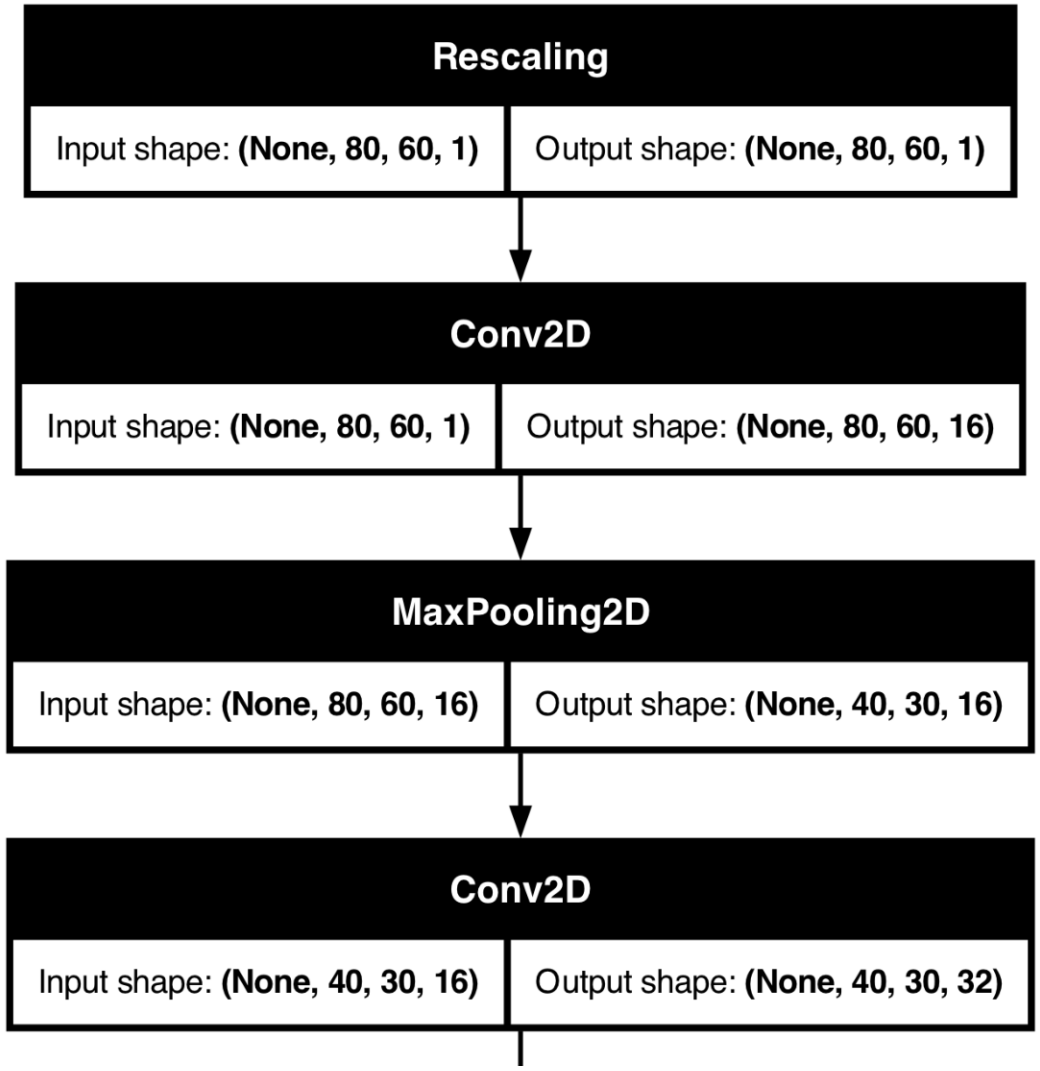
Model: "sequential_1"

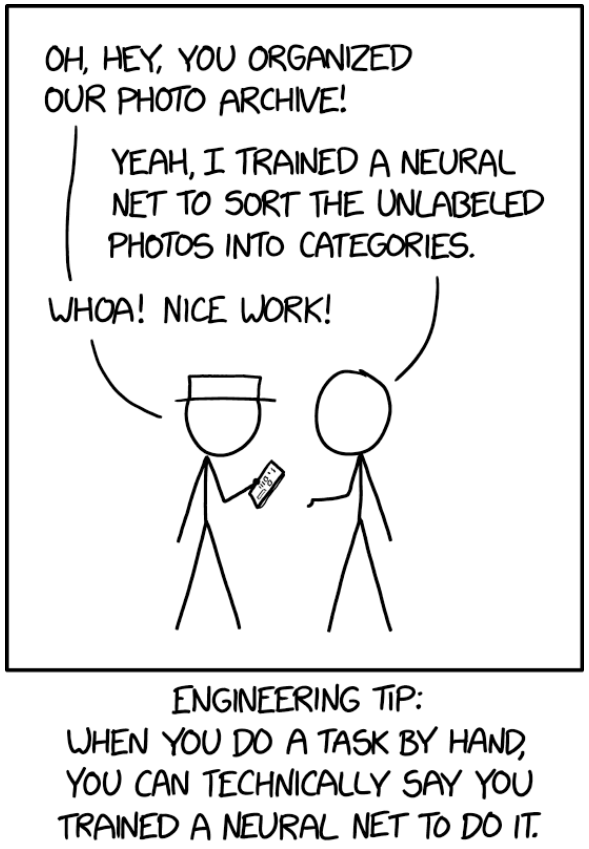
Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 80, 60, 1)	0
conv1 (Conv2D)	(None, 80, 60, 16)	160
pool1 (MaxPooling2D)	(None, 40, 30, 16)	0
conv2 (Conv2D)	(None, 40, 30, 32)	4,640
pool2 (MaxPooling2D)	(None, 20, 15, 32)	0
conv3 (Conv2D)	(None, 20, 15, 64)	18,496
pool3 (MaxPooling2D)	(None, 5, 3, 64)	0
flatten_1 (Flatten)	(None, 960)	0
dense_1 (Dense)	(None, 64)	61,504
dense_2 (Dense)	(None, 55)	3,575

Total params: 88,375 (345.21 KB)
Trainable params: 88,375 (345.21 KB)
Non-trainable params: 0 (0.00 B)

Plot the CNN

```
1 plot_model(model, show_shapes=True)
```





Source: Randall Munroe (2019), xkcd #2173: Trained a Neural Net.

Fit the CNN

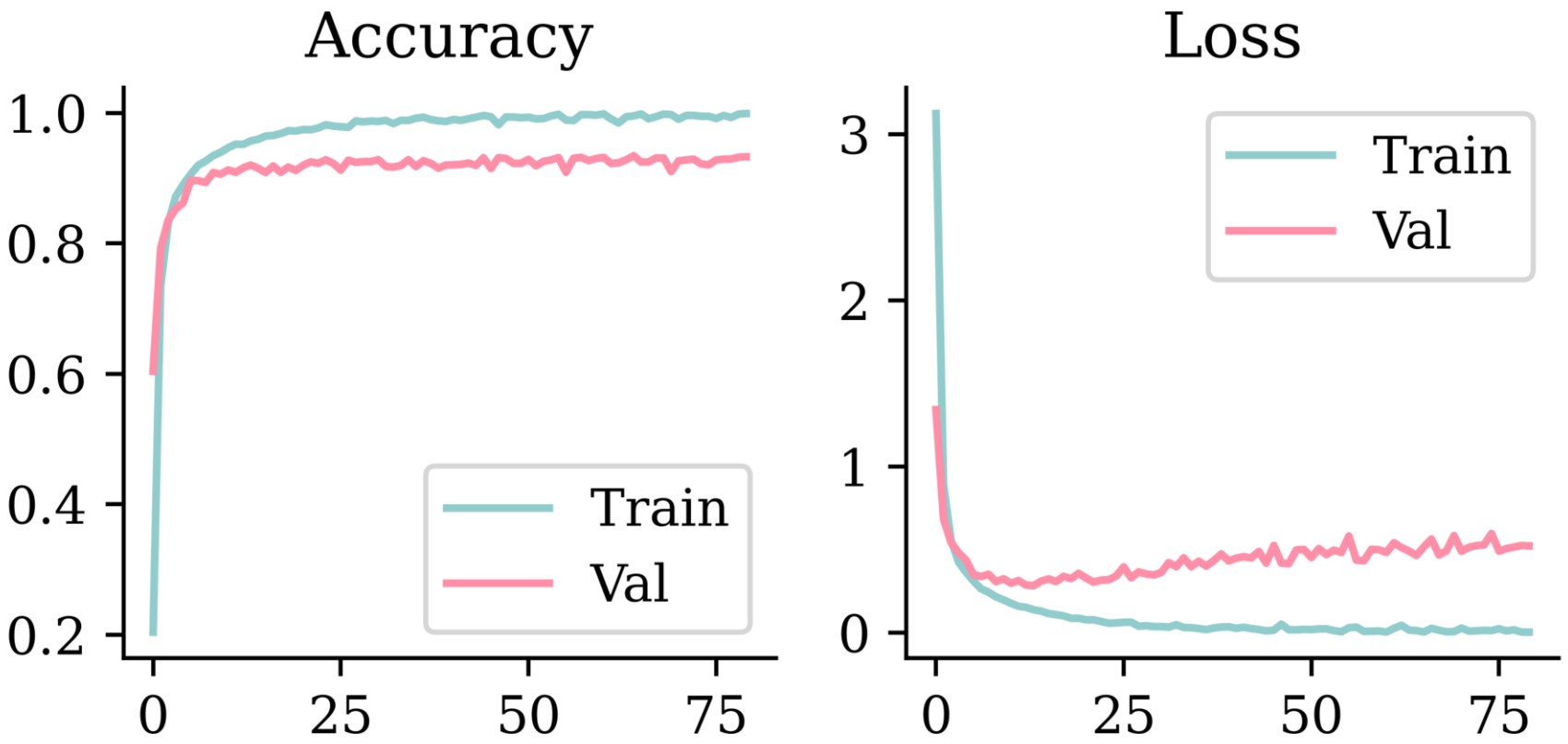
```
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)
topk = keras.metrics.SparseTopKCategoricalAccuracy(k=5)
model.compile(optimizer='adam', loss=loss, metrics=['accuracy', topk])

es = EarlyStopping(patience=15, restore_best_weights=True,
    monitor="val_accuracy", verbose=2)
hist = model.fit(X_train, y_train, validation_data=(X_val, y_val),
    epochs=100, batch_size=128, callbacks=[es], verbose=0)
history = hist.history
```

Tip

Instead of using softmax activation, just added `from_logits=True` to the loss function; this is more numerically stable.

Plot the loss/accuracy curves

```
1 plot_history(history)
```



Look at the metrics

```
print(model.evaluate(X_train, y_train, verbose=0))
print(model.evaluate(X_val, y_val, verbose=0))
```

```
[0.0042258091270923615, 0.9987967610359192, 1.0]
[0.46650248765945435, 0.9343371391296387, 0.9953430891036987]
```

```
loss_value, accuracy, top5_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Loss: {loss_value:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")
print(f"Test Top 5 Accuracy: {top5_accuracy:.4f}")
```

```
Test Loss: 0.9513
Test Accuracy: 0.8850
Test Top 5 Accuracy: 0.9875
```

Predict on the test set

Running `model.predict(X_test[0], verbose=0)` would crash. Why?

```
print(X_test[0].shape)
print(X_test[0][np.newaxis, :].shape)
print(X_test[[0]].shape)
```

```
(80, 60, 1)
(1, 80, 60, 1)
(1, 80, 60, 1)
```

```
model.predict(X_test[[0]], verbose=0)
```

```
array([[  39.32,  -42.6 ,  -76.12,  -46.71,  -15.65,  -21.84,  -80.2 ,
         -66.93,  -10.67,    0.46,  -30.95,  -42.12,  -40.14,  -48.4 ,
         -16.2 ,   -3.83,  -17.33,   25.23,   -6.01,  -48.1 ,  -52.31,
         -55.88, -112.03,  -40.44,  -27.87,  -36.71,  -38.15,   -9.84,
          -7.32,  -22.5 ,   -8.18,  -25.79,  -33.83,   10.17,  -27.51,
         -28.83,  -56.86,  -40.7 ,  -40.01,  -40.38,  -51.93,   -8.63,
          -9.4 ,   -0.41,   -2.17,   -8.25,  -19.06,  -11.19,  -74.19,
         -60.56,  -50.35,  -16.09,  -80.93,  -25.66,  -41.95]],
      dtype=float32)
```

UNSW SYDNEY

Predict on the test set II

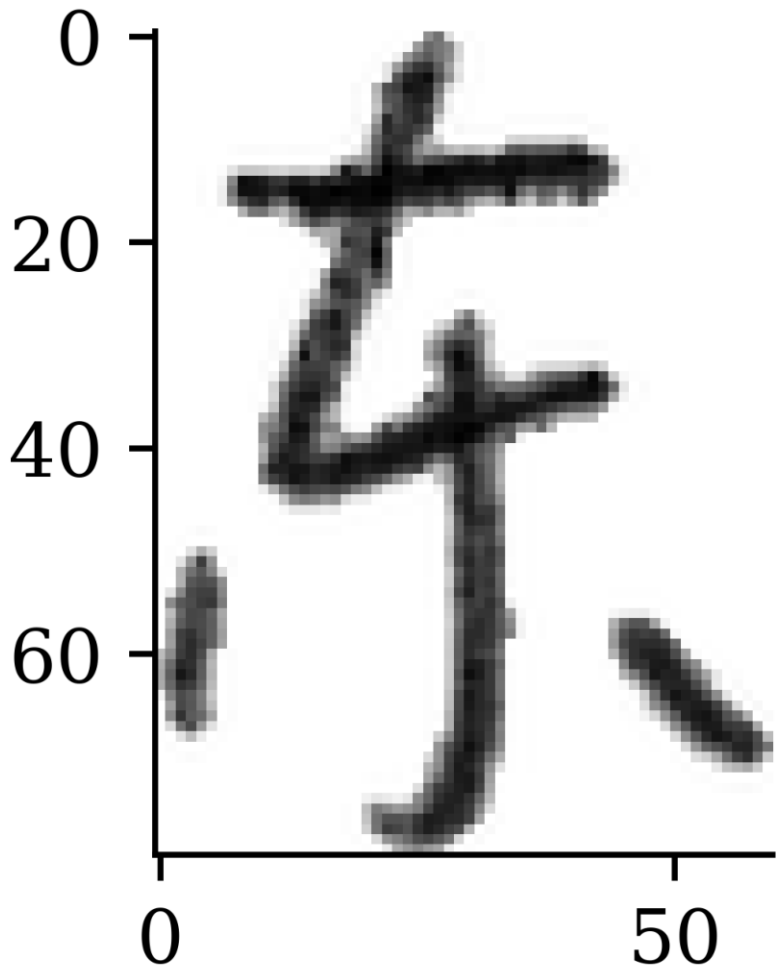
```
1 model.predict(X_test[[0]], verbose=0).arg
```

```
np.int64(0)
```

```
1 class_names[model.predict(X_test[[0]], ve
```

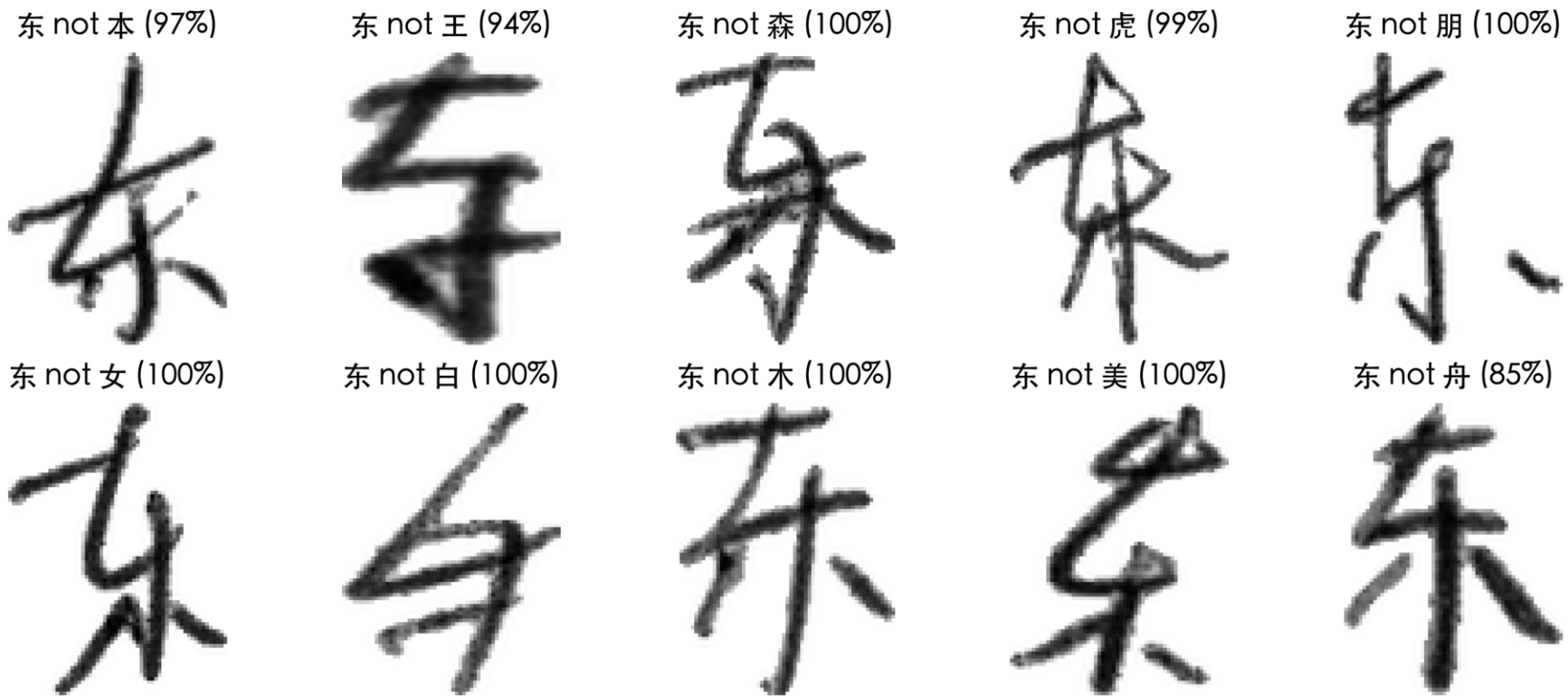
```
' '
```

```
1 plt.imshow(X_test[0], cmap="gray");
```



Take a look at the failure cases

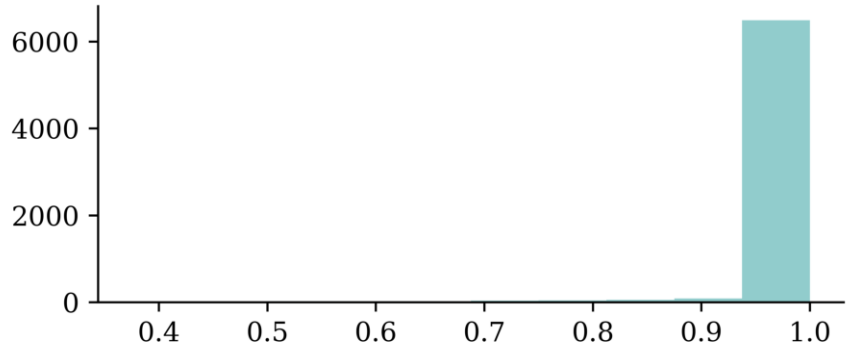
```
1 plot_failed_predictions(X_test, y_test, class_names)
```



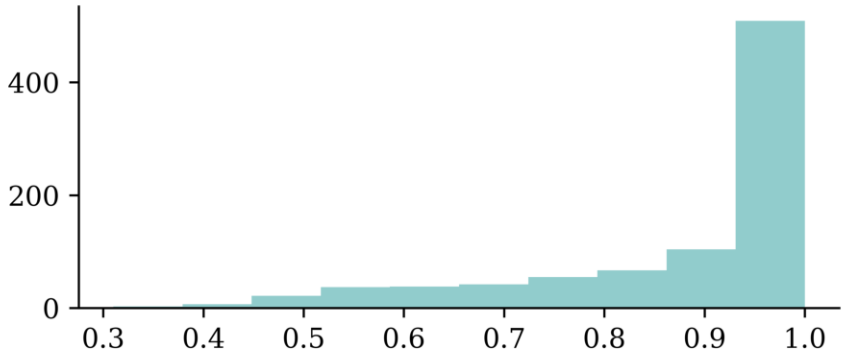
Confidence of predictions

```
y_log = model.predict(X_test, verbose=0)
y_pred = keras.ops.convert_to_numpy(keras.activations.softmax(y_log))
y_pred_class = np.argmax(y_pred, axis=1)
y_pred_prob = y_pred[np.arange(y_pred.shape[0]), y_pred_class]

confidence_when_correct = y_pred_prob[y_pred_class == y_test]
confidence_when_wrong = y_pred_prob[y_pred_class ≠ y_test]
```

```
plt.hist(confidence_when_correct);
```




```
plt.hist(confidence_when_wrong);
```



Lecture Outline

- Introduction
- The Convolution Operation
- Convolutional Neural Networks
- Chinese Character Recognition Dataset
- Training Our Models
- **Hyperparameter Optimisation**
- Transfer Learning

How do we pick the best hyperparameters?

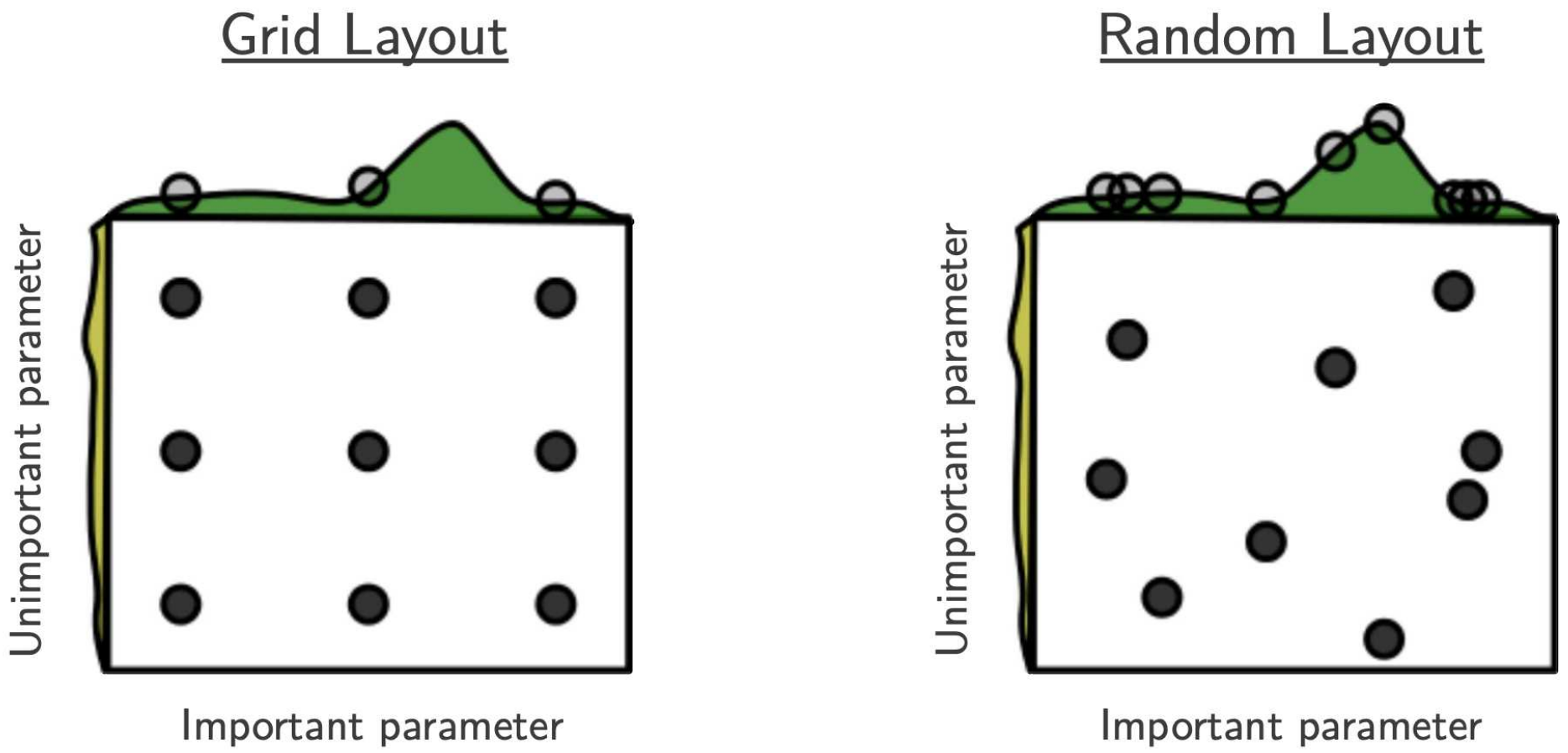
Make a range potential values for hyperparameter, and try many combinations of them.

> "hyper-parameter optimization should be regarded as a formal outer loop in the learning process"
>
> — Bergstra et al. (2011, p. 1).

> "HPO faces several challenges which make it a hard problem in practice:
>
> - Function evaluations can be extremely expensive for large models (e.g., in deep learning), complex machine learning pipelines, or large datesets.
> - The configuration space is often complex (comprising a mix of continuous, categorical and conditional hyperparameters) and high-dimensional. Furthermore, it is not always clear which of an algorithm's hyperparameters need to be optimized, and in which ranges.
> - We usually don't have access to a gradient of the loss function with respect to the hyperparameters."
>
> — Feurer & Hutter (2019, p. 4)

Could we just try every combination?

This technique, called grid search, would be too slow. Better to take random selections.



Source: Figure 1 directly from Bergstra & Bengio (2012).

Optuna

```python
def objective(trial):
    keras.utils.set_random_seed(trial.number)
    model = Sequential()
    model.add(Dense(
        trial.suggest_categorical("neurons", [4, 8, 16, 32, 64, 128, 256]),
        activation=trial.suggest_categorical("activation",
            ["relu", "leaky_relu", "tanh"]),
    ))
    model.add(Dense(1, activation="exponential"))

    learning_rate = trial.suggest_float("lr", 1e-4, 1e-2, log=True)
    opt = keras.optimizers.Adam(learning_rate=learning_rate)
    model.compile(optimizer=opt, loss="poisson")

    es = EarlyStopping(patience=3, restore_best_weights=True)
    model.fit(X_train_sc, y_train, epochs=100, batch_size=256,
        callbacks=[es], validation_data=(X_val_sc, y_val), verbose=0)

    return model.evaluate(X_val_sc, y_val, verbose=0)
```

Do a random search

```python
sampler = optuna.samplers.RandomSampler(seed=123)
study = optuna.create_study(direction="minimize", sampler=sampler)

study.optimize(objective, n_trials=10)
```

```python
print(f"Best value: {study.best_trial.value:.4f}")
print(f"Best params:")
for k, v in study.best_trial.params.items():
    print(f"  {k}: {v}")
```

```
Best value: 0.3188
Best params:
  neurons: 256
  activation: relu
  lr: 0.00048568650020512866
```

Recover the winning model

Because each trial seeded itself with its own trial number, we can rebuild the *exact* winning model from its saved parameters — no need to store the trained weights.

```
p = study.best_trial.params
keras.utils.set_random_seed(study.best_trial.number)

best_model = Sequential([
    Dense(p["neurons"], activation=p["activation"]),
    Dense(1, activation="exponential"),
])
best_model.compile(optimizer=keras.optimizers.Adam(p["lr"]), loss="poisson")

es = EarlyStopping(patience=3, restore_best_weights=True)
best_model.fit(X_train_sc, y_train, epochs=100, batch_size=256,
    callbacks=[es], validation_data=(X_val_sc, y_val), verbose=0)

print(f"Rebuilt loss: {best_model.evaluate(X_val_sc, y_val, verbose=0):.4f}")
print(f"Tuning best:  {study.best_trial.value:.4f}")
```

```
Rebuilt loss: 0.3188
Tuning best:  0.3188
```

Tune layers separately

```python
def objective(trial):
    keras.utils.set_random_seed(trial.number)
    model = Sequential()

    for i in range(trial.suggest_int("numHiddenLayers", 1, 3)):
        model.add(Dense(
            trial.suggest_categorical(f"neurons_{i}", [8, 16, 32, 64]),
            activation="relu"))
    model.add(Dense(1, activation="exponential"))

    opt = keras.optimizers.Adam(learning_rate=0.0005)
    model.compile(optimizer=opt, loss="poisson")

    es = EarlyStopping(patience=3, restore_best_weights=True)
    model.fit(X_train_sc, y_train, epochs=100, batch_size=256,
        callbacks=[es], validation_data=(X_val_sc, y_val), verbose=0)

    return model.evaluate(X_val_sc, y_val, verbose=0)
```

Do a Bayesian search

```python
sampler = optuna.samplers.TPESampler(seed=123, n_startup_trials=5)
study = optuna.create_study(direction="minimize", sampler=sampler)

study.optimize(objective, n_trials=10)
```

```python
print(f"Best value: {study.best_trial.value:.4f}")
print(f"Best params:")
for k, v in study.best_trial.params.items():
    print(f"  {k}: {v}")
```

```
Best value: 0.3140
Best params:
  numHiddenLayers: 3
  neurons_0: 64
  neurons_1: 32
  neurons_2: 32
```

Tip

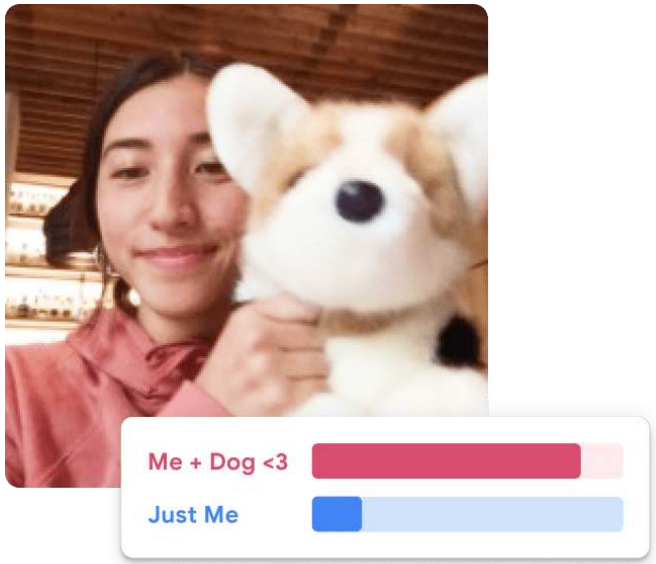
Neural networks are sensitive to their random initialisation: a hyperparameter combination that looks good (or bad) on a single run might just have been lucky. A more robust approach is to train 3–5 models with the same hyperparameters but different random seeds, and average the validation loss — selecting architectures that perform well *and* reliably.

See Bergstra et al. (2011, sec. 4) for details of the Tree-structured Parzen Estimator approach.

Lecture Outline

- Introduction
- The Convolution Operation
- Convolutional Neural Networks
- Chinese Character Recognition Dataset
- Training Our Models
- Hyperparameter Optimisation
- **Transfer Learning**

Demo: Object classification



Example object classification app

> "... these models use a technique called transfer learning. There's a pretrained neural network, and when you create your own classes, you can sort of picture that your classes are becoming the last layer or step of the neural net. Specifically, both the image and pose models are learning off of pretrained mobilenet models ..."

Source: Teachable Machine FAQ

Keras Applications

Available models

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8

A catalogue of pretrained models at https://keras.io/api/applications/

Each has its own preprocess function that you need to apply to new inputs.

Pretrained model

```python
def classify_imagenet(paths, model_module, ModelClass, dims):
    images = [keras.utils.load_img(path, target_size=dims) for path in paths]
    image_array = np.array([keras.utils.img_to_array(img) for img in images])
    inputs = model_module.preprocess_input(image_array)

    model = ModelClass(weights="imagenet")
    Y_proba = model(inputs)
    top_k = model_module.decode_predictions(Y_proba, top=3)

    for image_index in range(len(images)):
        print(f"Image #{image_index}:")
        for class_id, name, y_proba in top_k[image_index]:
            print(f" {class_id} - {name} {int(y_proba*100)}%")
        print()
```

Predicted classes (MobileNet)

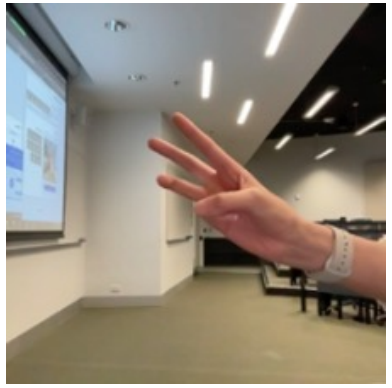
```
Image #0:
 n04399382 - teddy 89%
 n04254120 - soap_dispenser 7%
 n04462240 - toyshop 2%

Image #1:
 n03075370 - combination_lock 30%
 n04019541 - puck 26%
 n03666591 - lighter 10%

Image #2:
 n04009552 - projector 20%
 n03908714 - pencil_sharpener 17%
 n02951585 - can_opener 9%
```

Predicted classes (InceptionV3)

```
Image #0:
 n04399382 - teddy 87%
 n04162706 - seat_belt 2%
 n04462240 - toyshop 2%

Image #1:
 n04023962 - punching_bag 13%
 n03337140 - file 7%
 n02992529 - cellular_telephone 3%

Image #2:
 n04005630 - prison 4%
 n03337140 - file 4%
 n06596364 - comic_book 2%
```

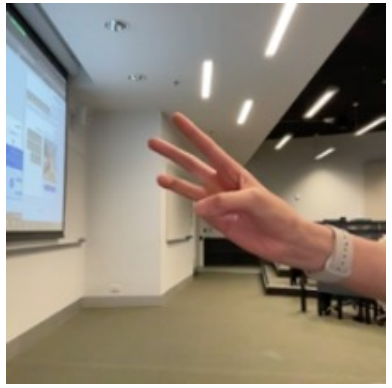
Predicted classes III (MobileNet)

```
Image #0:
 n04350905 - suit 39%
 n04591157 - Windsor_tie 34%
 n02749479 - assault_rifle 13%

Image #1:
 n03529860 - home_theater 25%
 n02749479 - assault_rifle 9%
 n04009552 - projector 5%

Image #2:
 n03529860 - home_theater 9%
 n03924679 - photocopier 7%
 n02786058 - Band_Aid 6%
```



Predicted classes III (InceptionV3)

```
Image #0:
 n04350905 - suit 25%
 n04591157 - Windsor_tie 11%
 n03630383 - lab_coat 6%

Image #1:
 n04507155 - umbrella 52%
 n04404412 - television 2%
 n03529860 - home_theater 2%

Image #2:
 n04404412 - television 17%
 n02777292 - balance_beam 7%
 n03942813 - ping-pong_ball 6%
```



Transfer learned model

```
model_file = "converted_keras/keras_model.h5"
model = keras.models.load_model(model_file)
```

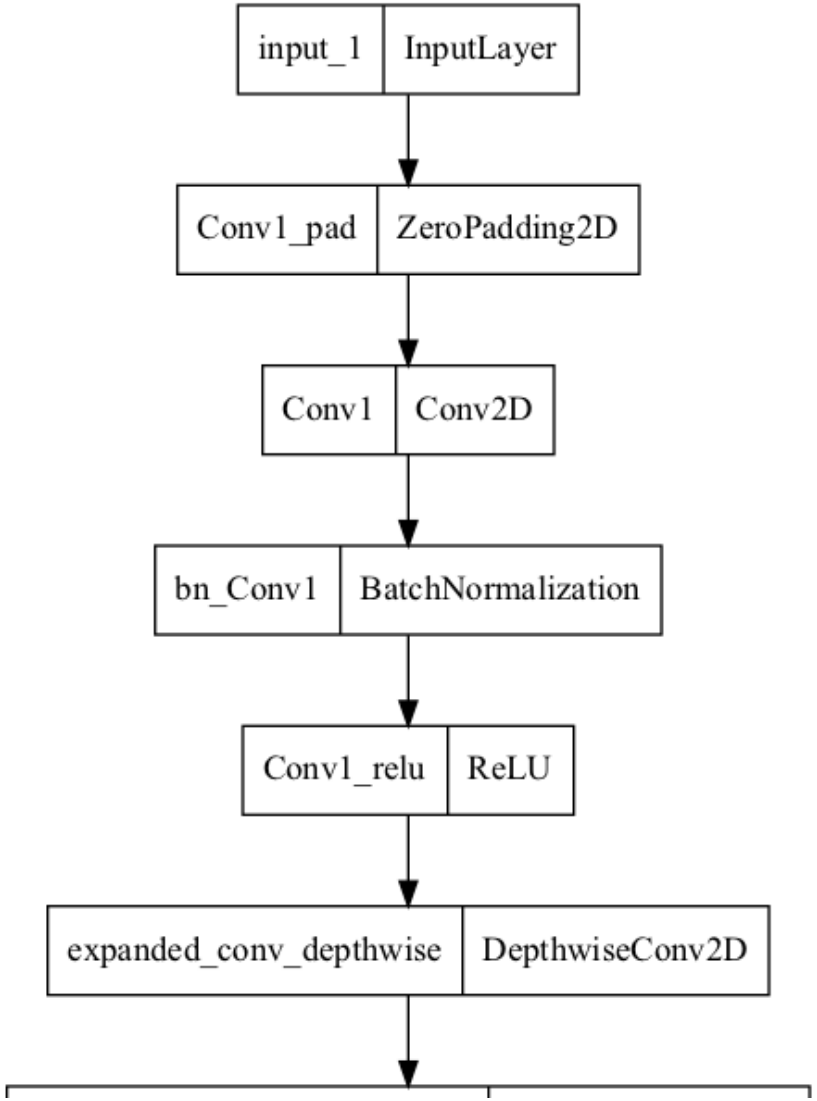
```
model.layers[0].layers[0].layers
```

```
[<InputLayer name=input_1, built=True>,
<ZeroPadding2D name=Conv1_pad, built=True>,
<Conv2D name=Conv1, built=True>,
<BatchNormalization name=bn_Conv1, built=True>,
<ReLU name=Conv1_relu, built=True>,
<DepthwiseConv2D name=expanded_conv_depthwise, built=True>,
<BatchNormalization name=expanded_conv_depthwise_BN, built=True>,
<ReLU name=expanded_conv_depthwise_relu, built=True>,
<Conv2D name=expanded_conv_project, built=True>,
<BatchNormalization name=expanded_conv_project_BN, built=True>,
<Conv2D name=block_1_expand, built=True>,
<BatchNormalization name=block_1_expand_BN, built=True>,
<ReLU name=block_1_expand_relu, built=True>,
<ZeroPadding2D name=block_1_pad, built=True>,
<DepthwiseConv2D name=block_1_depthwise, built=True>,
<BatchNormalization name=block_1_depthwise_BN, built=True>,
```

```
len(model.layers[0].layers[0].layers)
```

```
155
```

The original pretrained model



Transfer learning

```python
# Pull in the base model we are transferring from.
base_model = keras.applications.Xception(
    weights="imagenet",  # Load weights pre-trained on ImageNet.
    input_shape=(150, 150, 3),
    include_top=False,
)  # Discard the ImageNet classifier at the top.

# Tell it not to update its weights.
base_model.trainable = False

# Make our new model on top of the base model.
inputs = Input(shape=(150, 150, 3))
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
outputs = Dense(1)(x)
model = Model(inputs, outputs)

# Compile and fit on our data.
model.compile(
    optimizer=keras.optimizers.Adam(),
    loss=keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[keras.metrics.BinaryAccuracy()],
)
model.fit(new_dataset, epochs=2, callbacks=..., validation_data=...)
```

Source: François Chollet (2020), Transfer learning & fine-tuning, Keras documentation.

Fine-tuning

```python
# Unfreeze the base model
base_model.trainable = True

# It's important to recompile your model after you make any changes
# to the `trainable` attribute of any inner layer, so that your changes
# are take into account
model.compile(
    optimizer=keras.optimizers.Adam(1e-5),  # Very low learning rate
    loss=keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=[keras.metrics.BinaryAccuracy()],
)

# Train end-to-end. Be careful to stop before you overfit!
model.fit(new_dataset, epochs=1, callbacks=..., validation_data=...)
```

Caution

Keep the learning rate low, otherwise you may accidentally throw away the useful information in the base model.

Source: François Chollet (2020), Transfer learning & fine-tuning, Keras documentation.

Package Versions

```
1 from watermark import watermark
2 print(watermark(python=True, packages="keras,matplotlib,numpy,pandas,seaborn,scipy,torch"
```

```
Python implementation: CPython
Python version       : 3.14.5
IPython version      : 9.13.0

keras     : 3.14.1
matplotlib: 3.10.9
numpy     : 2.4.4
pandas    : 3.0.2
seaborn   : 0.13.2
scipy     : 1.17.1
torch     : 2.11.0
```

Glossary

- AlexNet
- channels
- computer vision
- convolutional layer
- convolutional network
- filter
- ImageNet challenge
- fine-tuning
- flatten layer
- kernel
- max pooling
- MNIST
- stride
- tensor (rank)
- transfer learning

References

Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems*, *24*.

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, *13*(2).

Feurer, M., & Hutter, F. (2019). Hyperparameter optimization. In *Automated machine learning: Methods, systems, challenges* (pp. 3–33). Springer.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Glassner, A. (2021). *Deep learning: A visual approach*. No Starch Press.

Goodfellow, I. J., Bulatov, Y., Ibarz, J., Arnoud, S., & Shet, V. (2014). Multi-digit number recognition from street view imagery using deep convolutional neural networks. *arXiv Preprint arXiv:1312.6082*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. 770–778.

Liu, C.-L., Yin, F., Wang, D.-H., & Wang, Q.-F. (2011). CASIA online and offline chinese handwriting databases. *2011 International Conference on Document Analysis and Recognition*, 37–41.